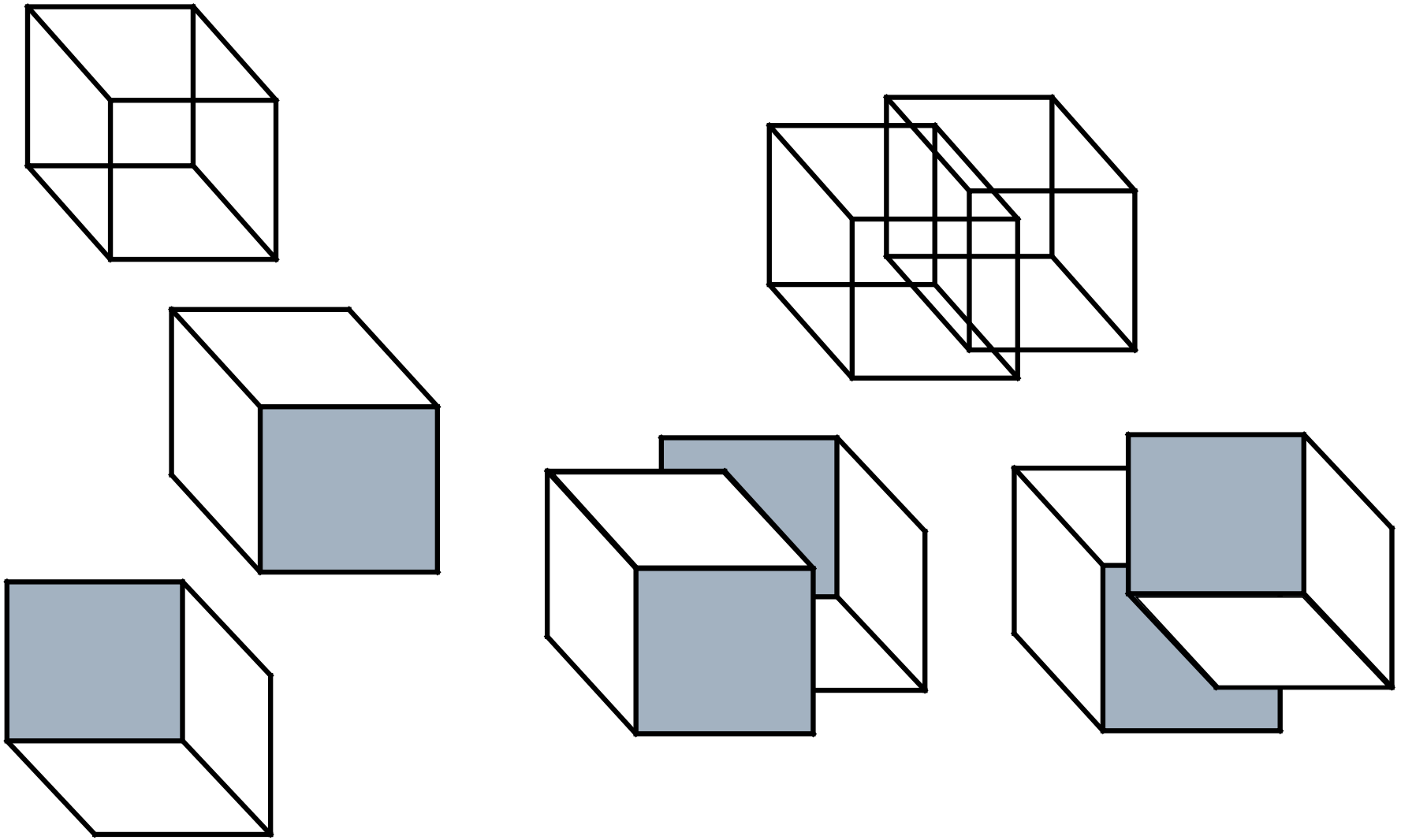
Hidden line and surface removal algorithms (1)

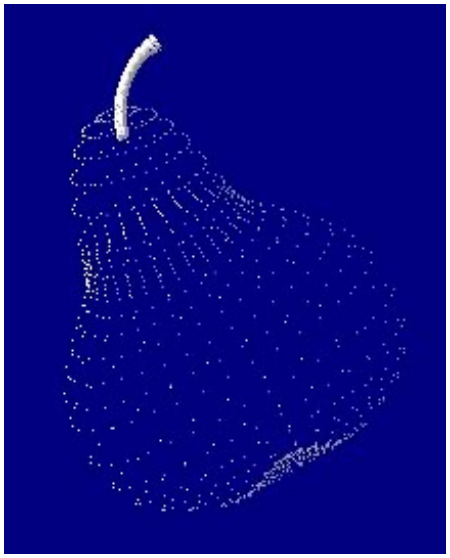
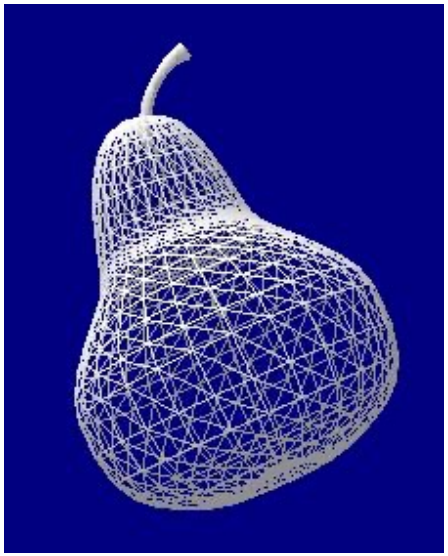
Contents

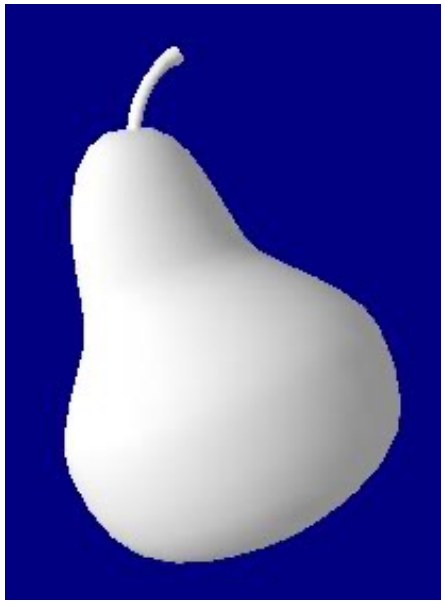
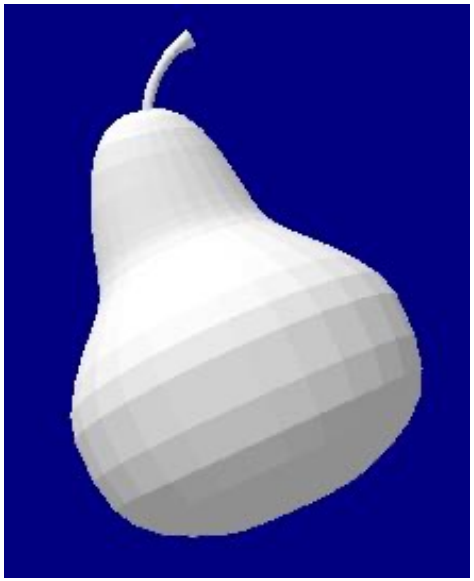
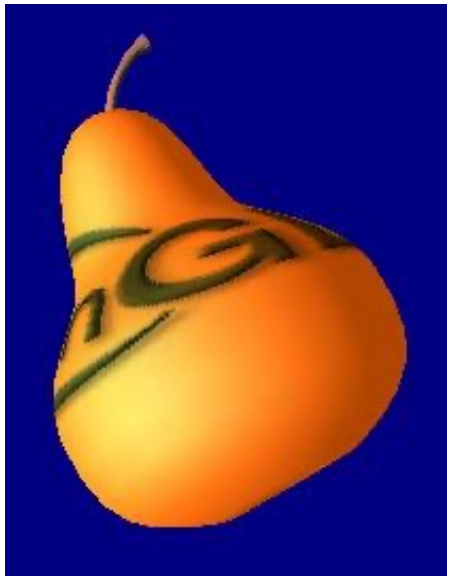
- Introduction
- Hidden line and surface removal algorithms
  1. **Floating horizon algorithm**
  2. **Back-face culling**
  3. **Area subdivision algorithm**
  4. **Depth buffer algorithms**
  5. Depth sorting algorithms
  6. Scan-line algorithms
  7. Octree algorithm
  8. Ray casting algorithms
  9. BSP trees

Introduction

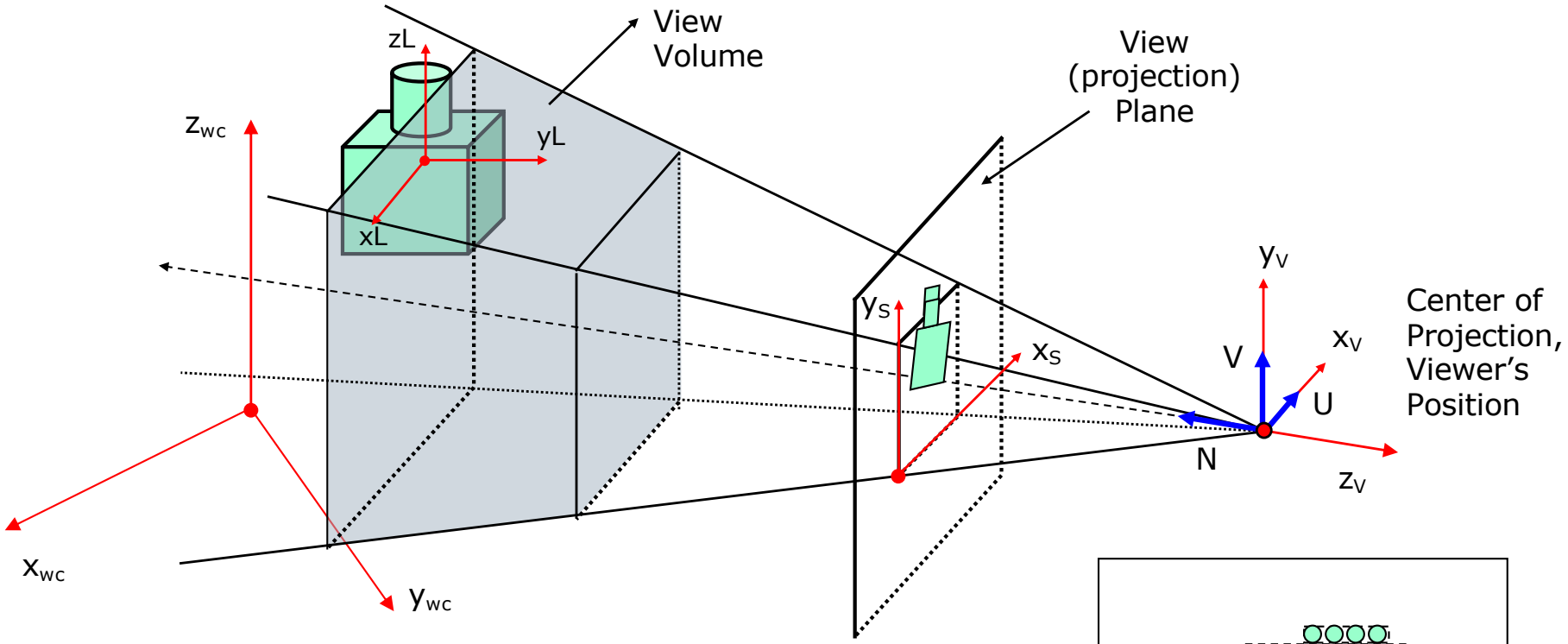


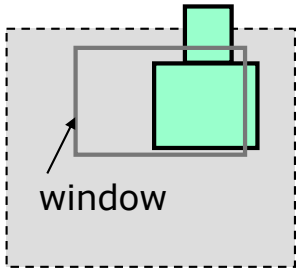
Introduction

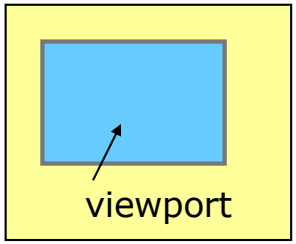




Conceptual model of 3D viewing process

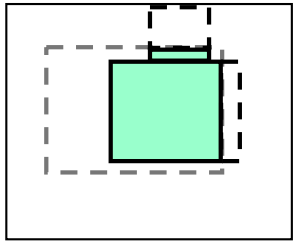




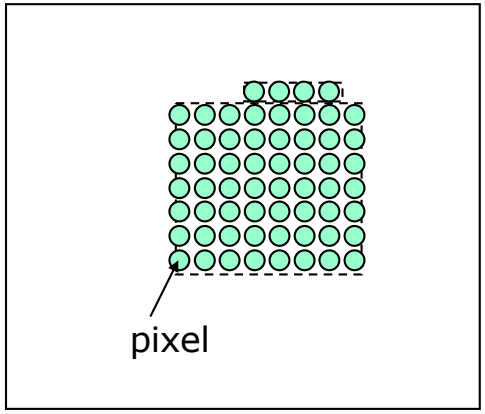
Window definition
in projection plane



Viewport definition
in projection plane

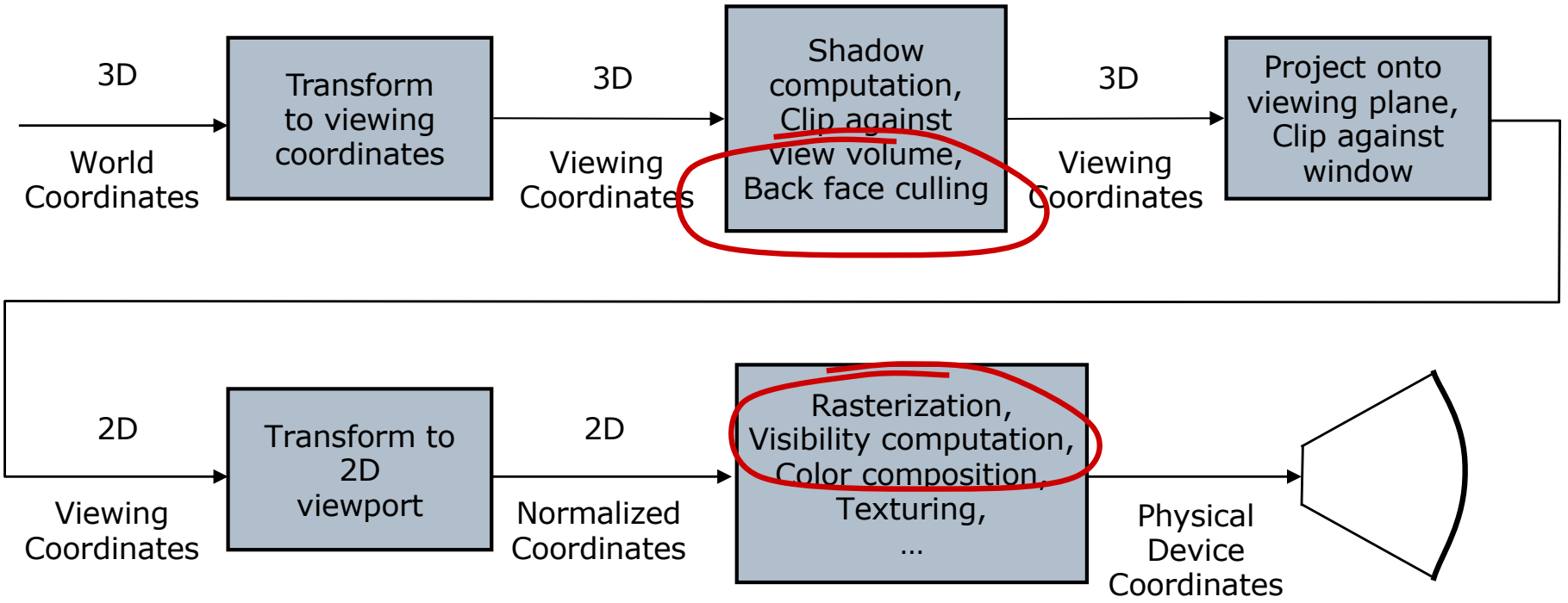


Window's content
inside the viewport

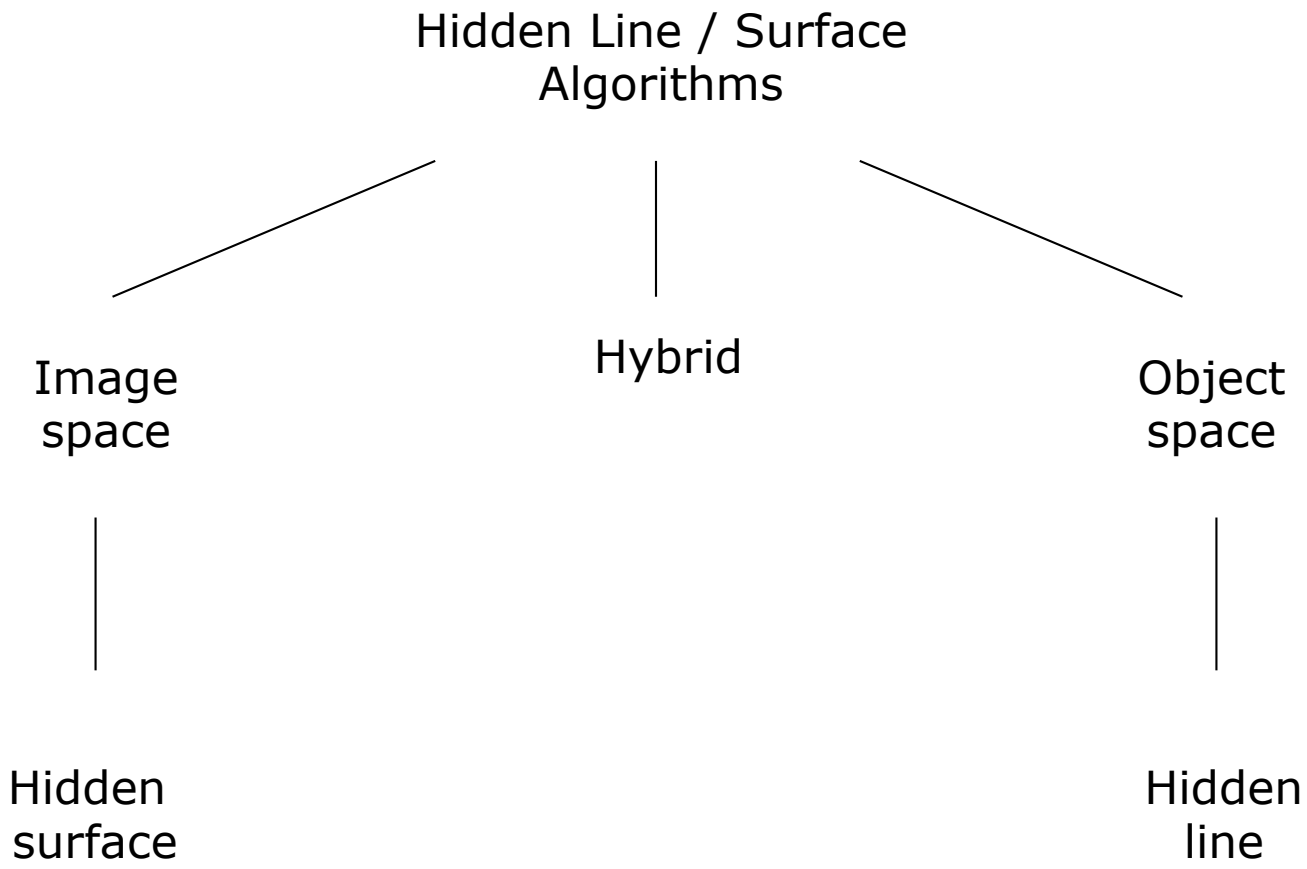


Scan conversion onto the
raster screen

Viewing transformations pipeline

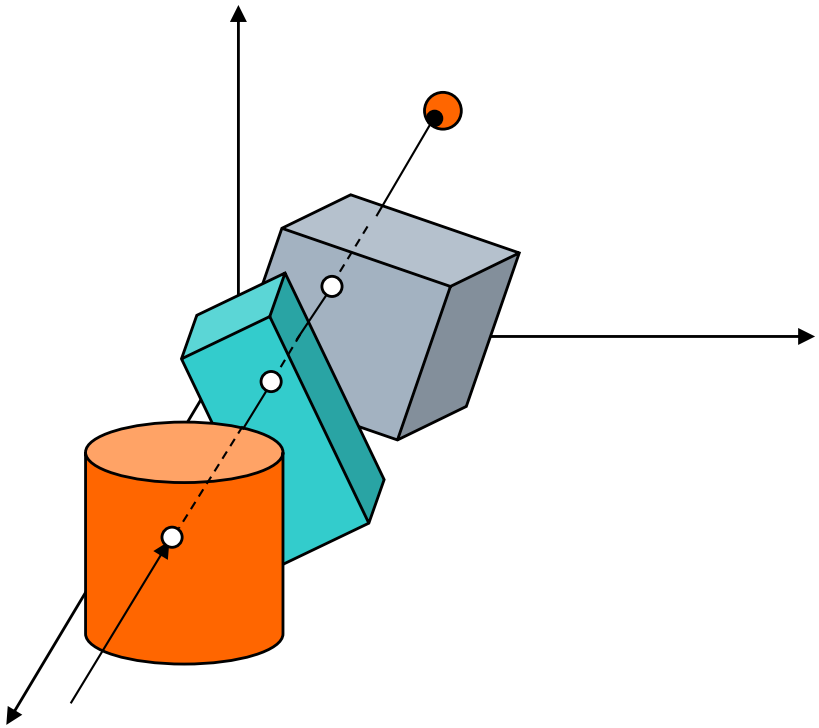


Taxonomy of algorithms



Hidden line and surface removal

- Algorithms in
  1. Object space
     - Determine if an element (point, polygon, surface, etc) is visible
  2. Image space
     - Determine what is visible on every pixel



for *each object in the world* **{**

determines those parts of the object whose view is unobstructed by other parts of it or any other objects;

draw those parts in the appropriate color;

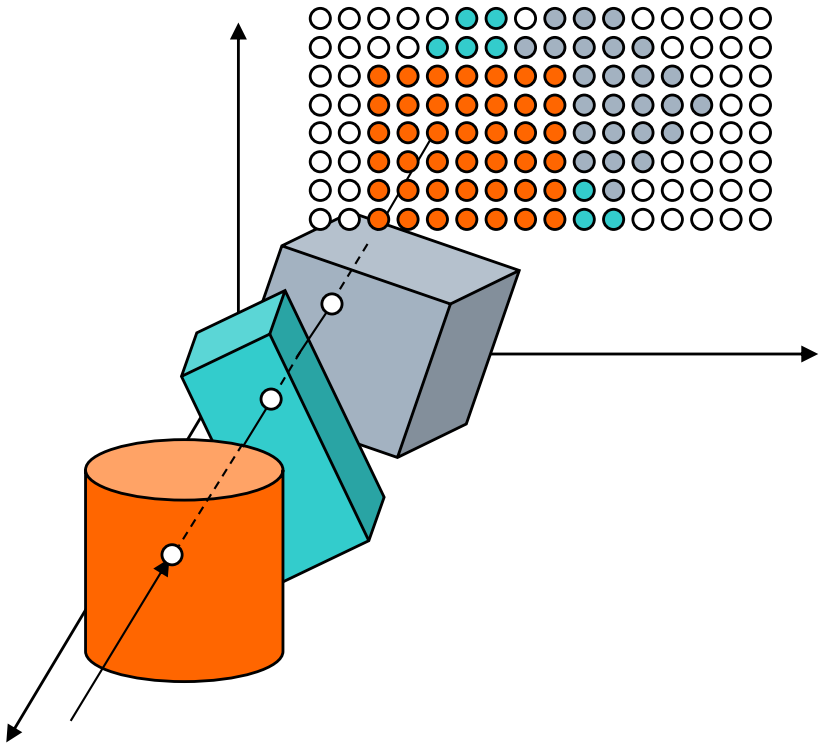
}

Computational effort: n^2

n: number of the objects

Hidden line and surface removal

- Algorithms in
  1. Object space
     - Determine if an element (point, polygon, surface, etc) is visible
  2. Image space
     - Determine what is visible on every pixel



for *each pixel in the image* **{**

determines the object closest to the viewer that is projected onto the pixel;

draw the pixel in the appropriate color;

}

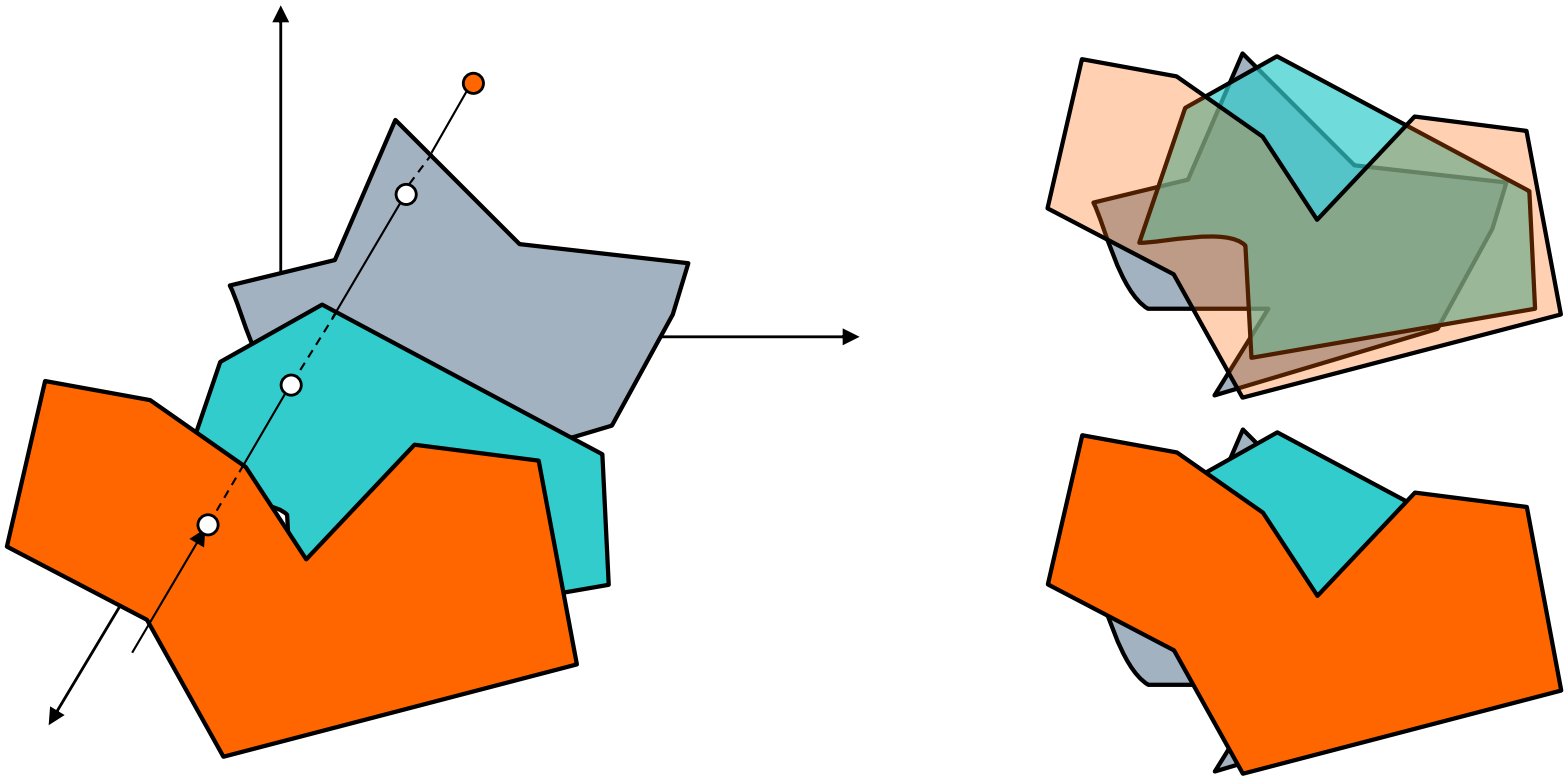
Computational effort: np

n: number of the objects

P: number of pixels

Basic tools

- Sorting
  - Points, polygons, edges, vertices etc.
- Ordered drawing
  - Over drawing into the image buffer.



Sorting efficiency

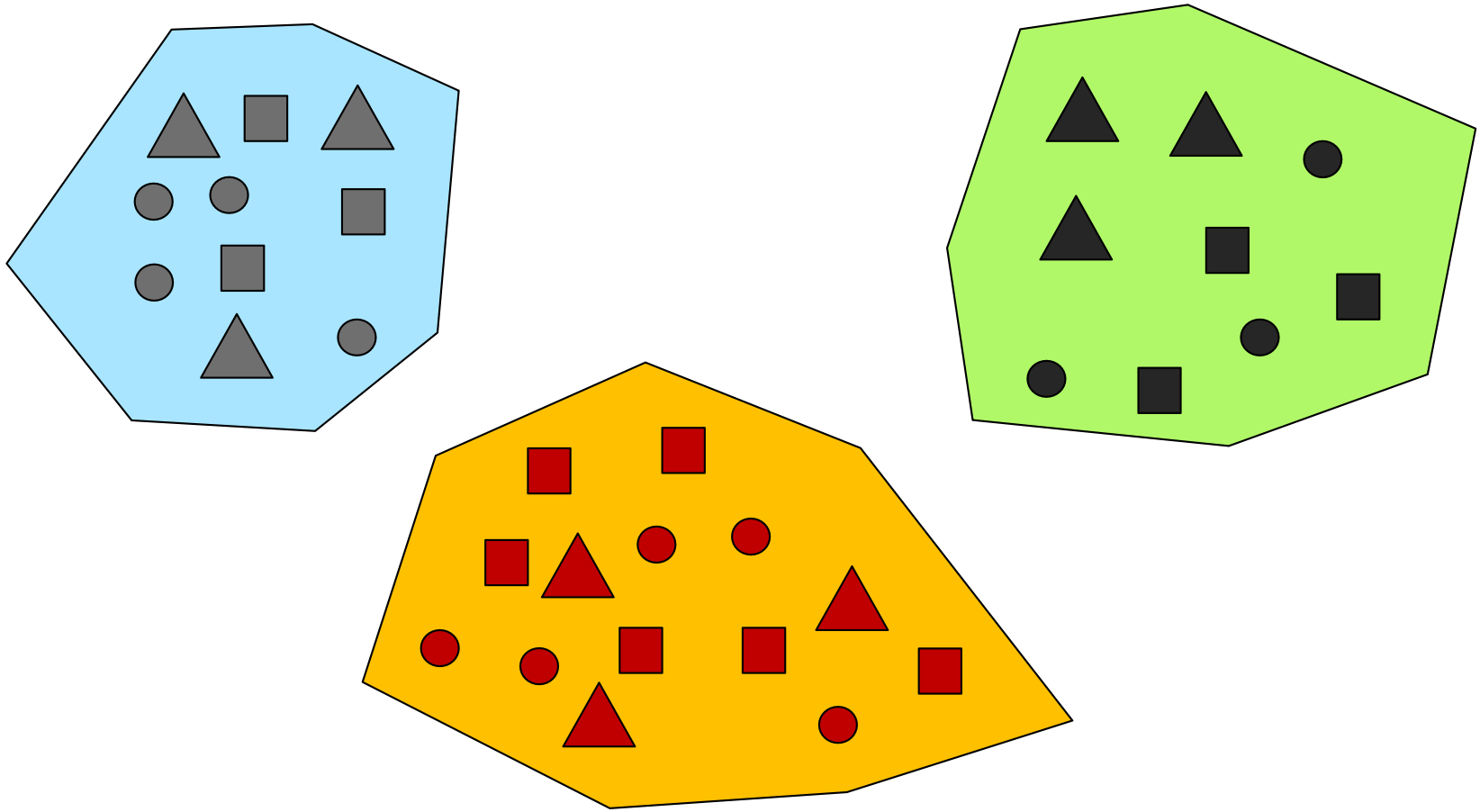
1. Coherence
   - On scan line, on edge, on surface, on polygon, area, depth, etc.
2. Depth sort simplification
   - Perspective transformation
   - Keep depth information
   - Orthogonal vs perspective projection
3. 2D and 3D extents

Coherence

- Object coherence
  - If one object is entirely separate from another, comparisons may need to be done only between the two objects, and not between their component faces or edges.
- Face coherence
  - Surface properties typically vary smoothly across a face, allowing computations for one part of a face to be modified incrementally to apply to adjacent parts.
- Edge coherence
  - An edge may change visibility only where it crosses behind a visible edge or penetrates a visible face.
- Scan-line coherence
  - The set of visible object spans determined for one scan line of an image typically differs little from the set on the previous line.
- Depth coherence
  - Adjacent parts of the same surface are typically close in depth. Once the depth at one point of the surface is calculated, the depth of points on the rest of the surface can often be determined by a simple difference equation.
- Frame coherence
  - Pictures at two successive points in time are likely to be quite similar, despite small changes in objects and viewpoint.

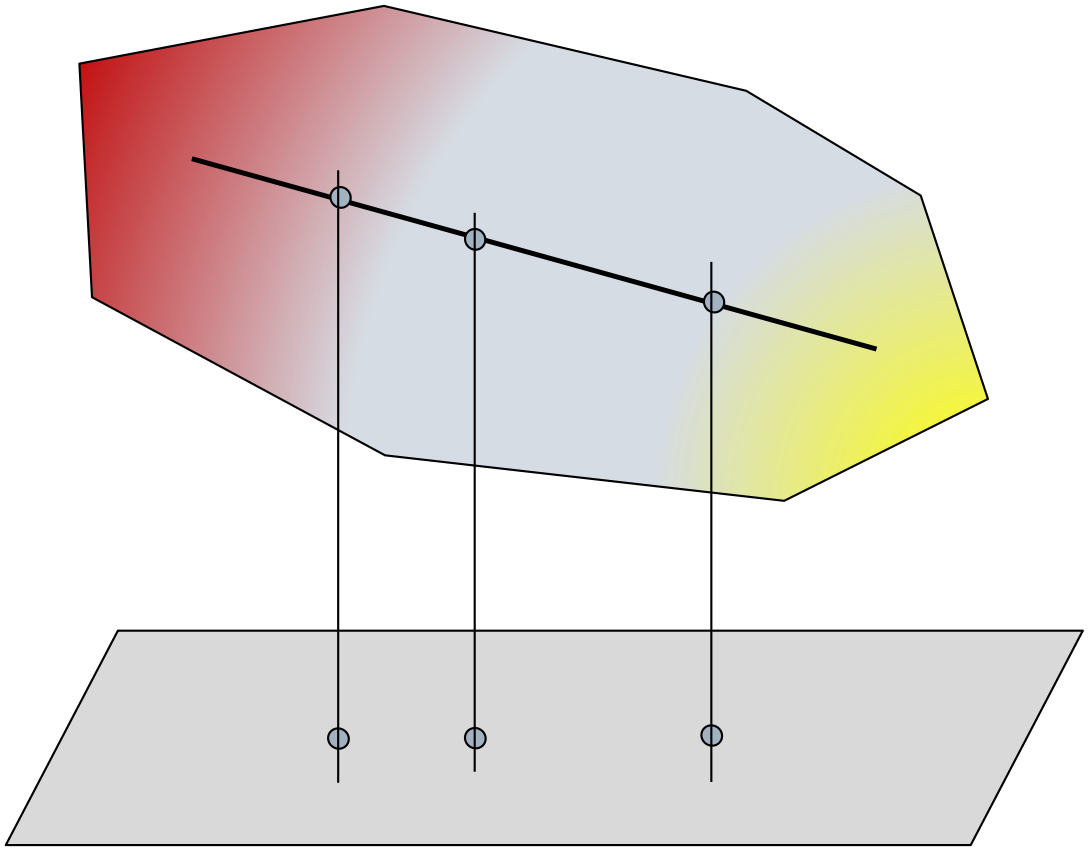
Object coherence - example

Object coherence: If one object is entirely separate from another, comparisons may need to be done only between the two objects, and not between their component faces or edges.



Face coherence - example

Face coherence: surface properties typically vary smoothly across a face, allowing computations for one part of a face to be modified incrementally to apply to adjacent parts.



Edge coherence - example

$y = mx + b$

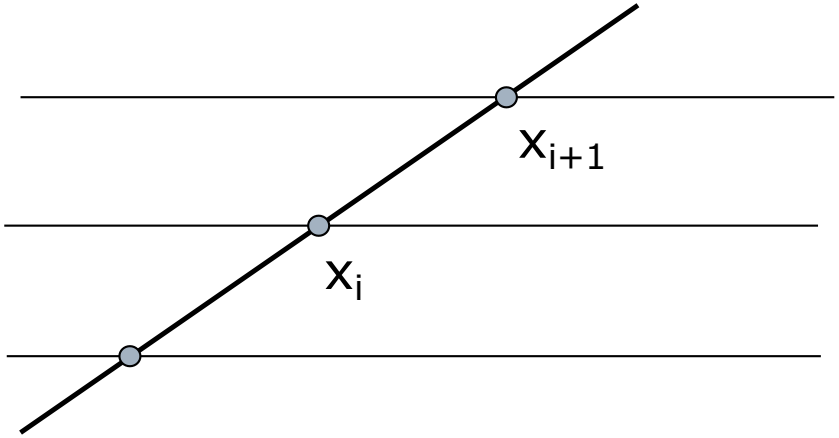
$y_i = mx_i + b$

$y_{i+1} = mx_{i+1} + b$

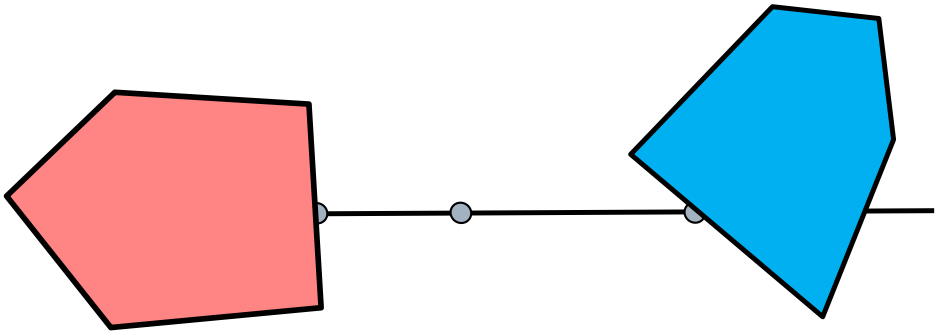
$x_{i+1} = x_i + (y_{i+1} - y_i)/m$

if $y_{i+1} - y_i = 1$ (const)

then $x_{i+1} = x_i + 1/m$ (const)

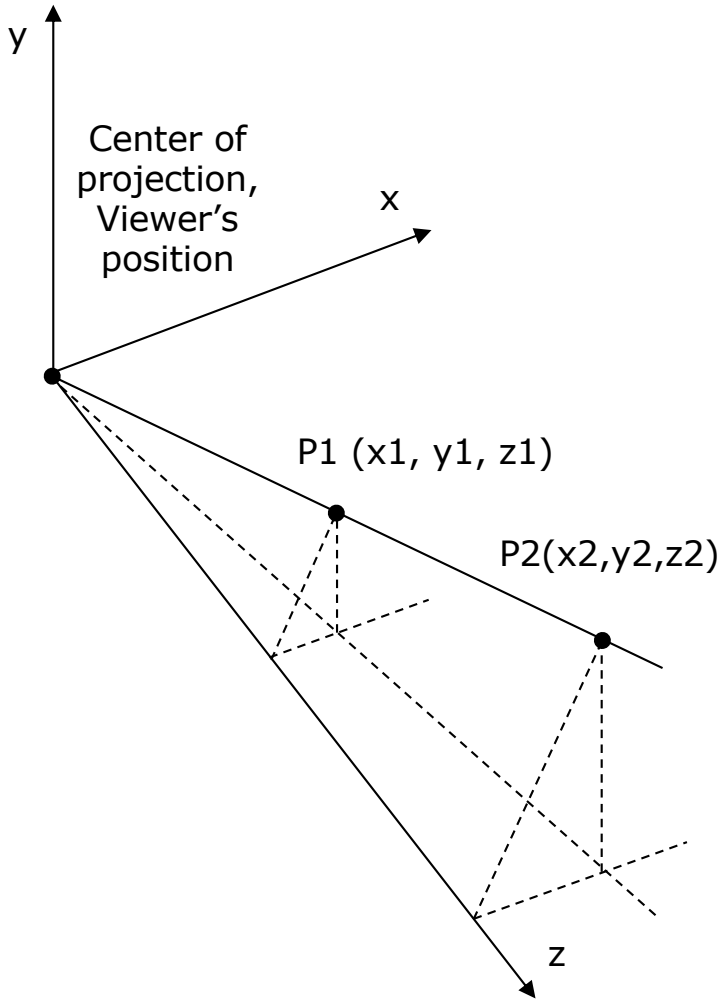


Coherence: till the intersection is on the edge, it is computed by adding a constant.



Coherence: an edge may change visibility only where it crosses behind a visible edge or penetrates a visible face.

Visibility in perspective projection



P1 and P2 are collinear, if:

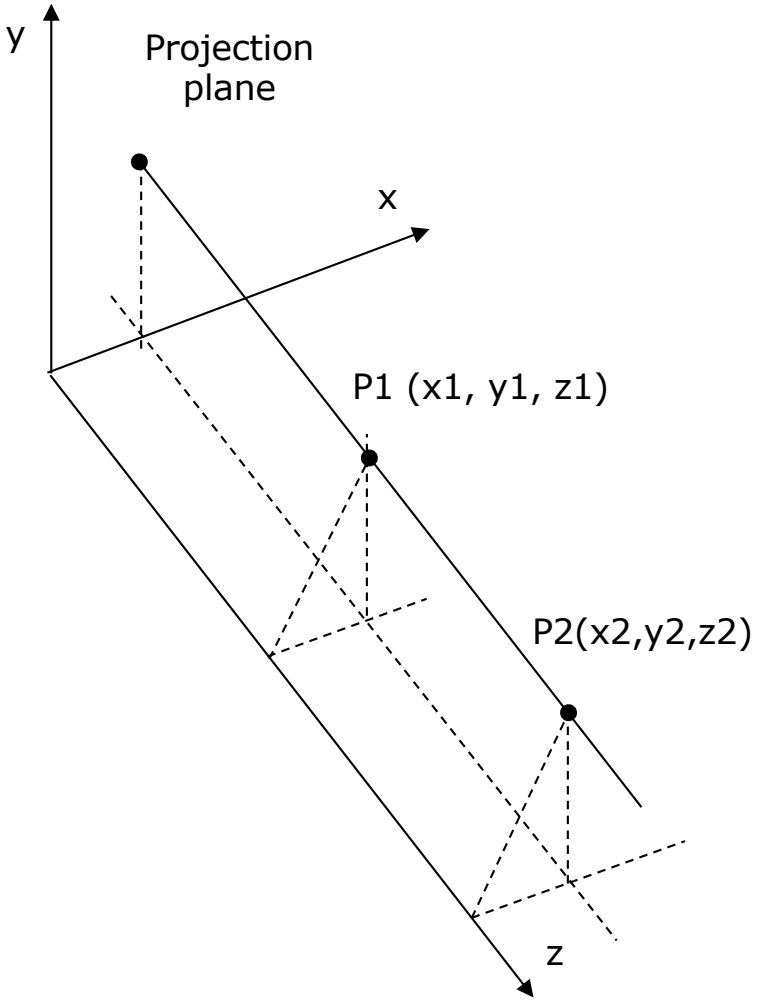
$x1/x2 = y1/y2 = z1/z2=k$ (const)

Checking a point P requires:

$x/x2 = y/y2 = z/z2 = k$

It needs to avoid expensive computation (multiplication, division, …)

Visibility in orthogonal projection



P1 and P2 collinear, if:

x1 = x2 and y1 = y2

Checking a point P requires:

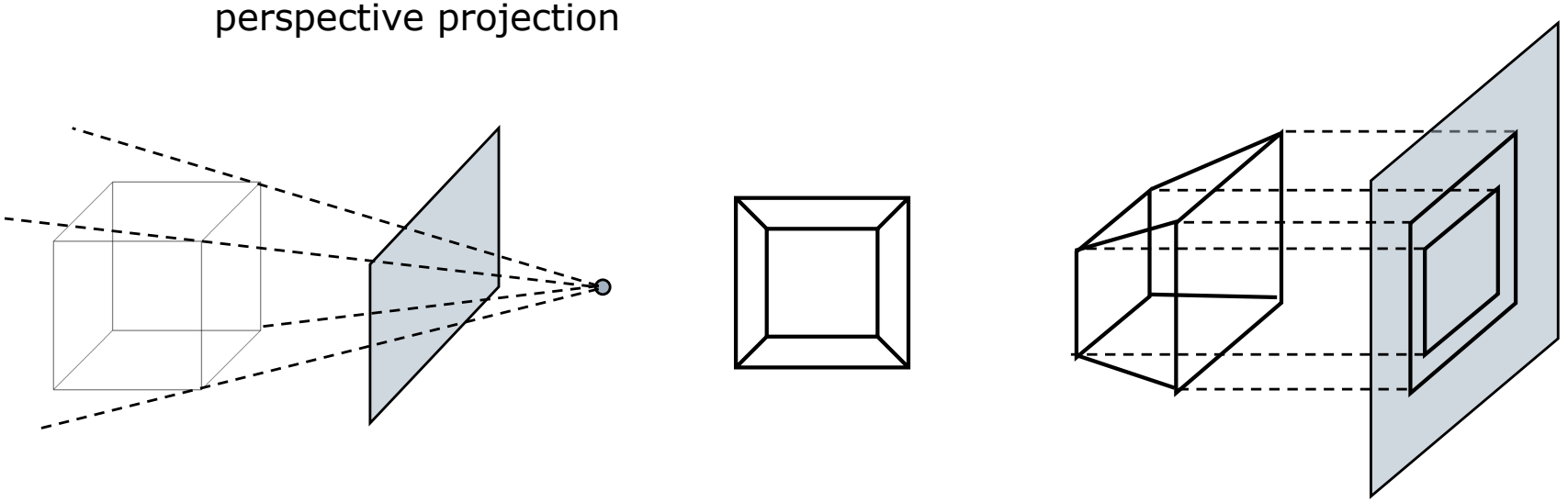
x = x2 and y = y2

No multiplication, division, …

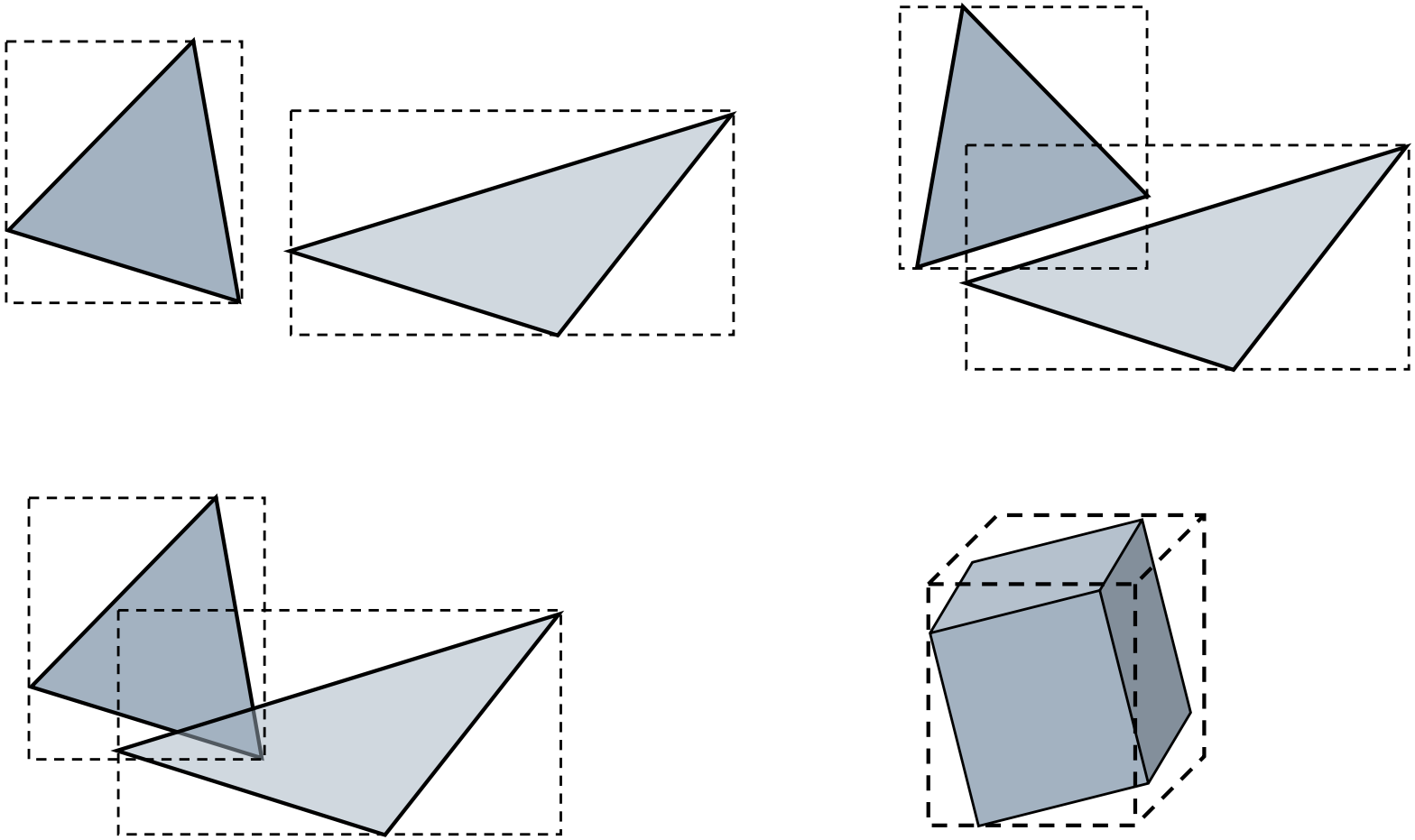
just comparisons.

Visibility and projections

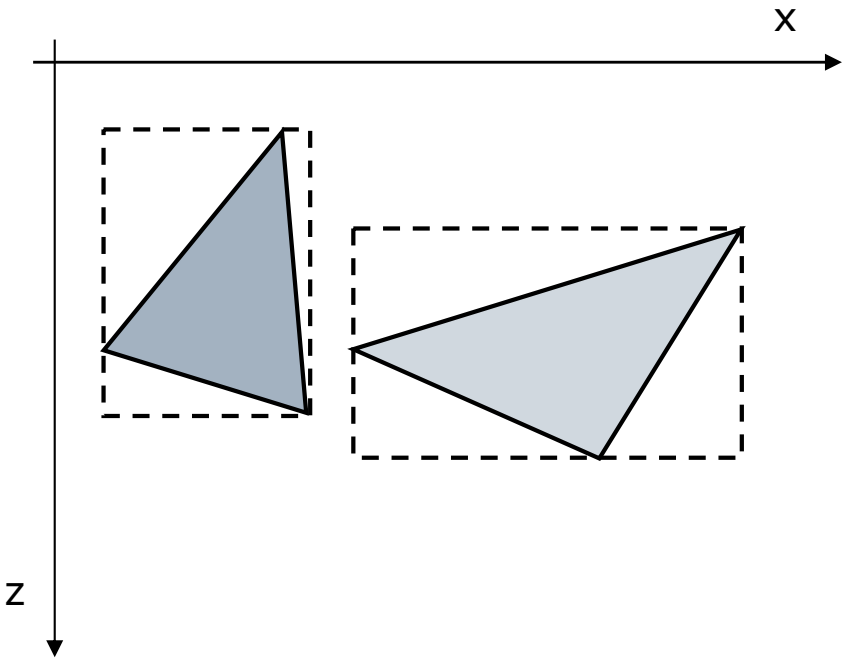
- The computation of the visibility requires:
  - Orthogonal projection
  - The visibility related computation needs depth information (z coordinates)
  - Keep depth information
- Example: frustum perspective transformation
  - Projection along the frustum (viewing volume) transforms the perspective projection into orthogonal projection
  - Keeps the z coordinate
  - Keeps depth order
  - Deform the object shape (e.g., cube into pyramidal frustum) in order to have through the orthogonal projection the same 2D image as through the perspective projection

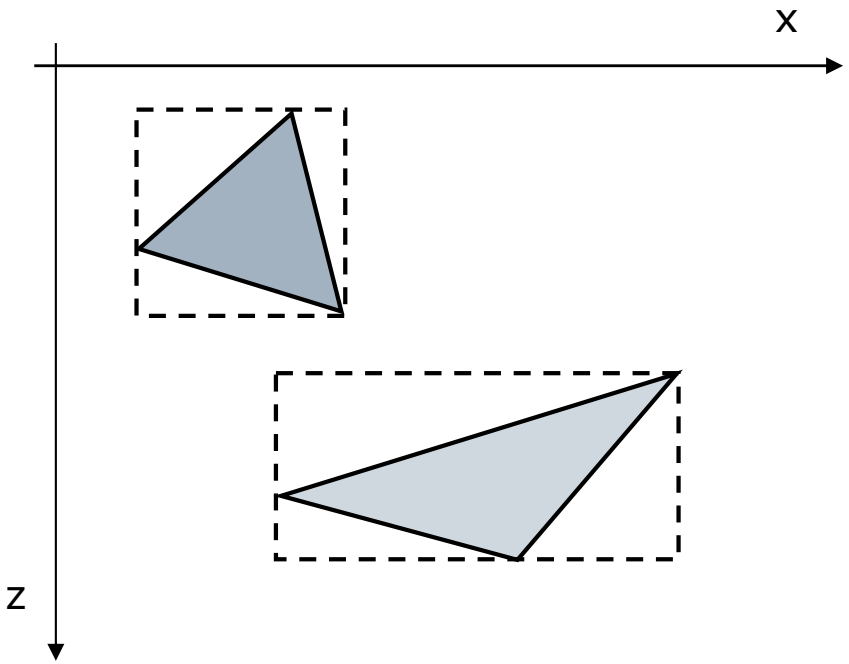


Object extents



Object extents





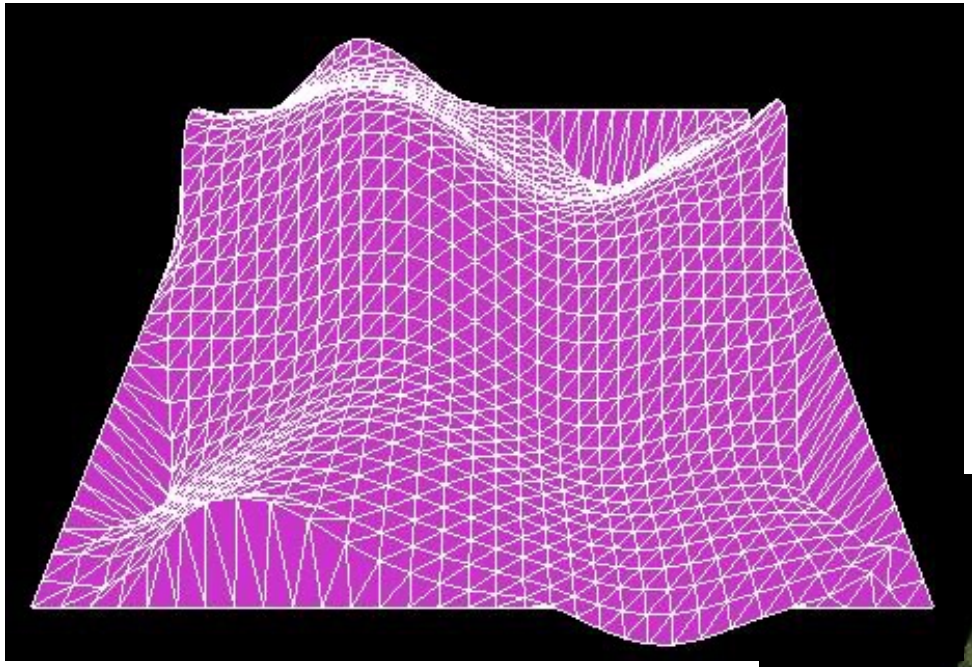
$x_{min2} \geq x_{max1}$

$x_{min1} \geq x_{max2}$

$z_{min2} \geq z_{max1}$

$z_{min1} \geq z_{max2}$

Hidden surface algorithms - example





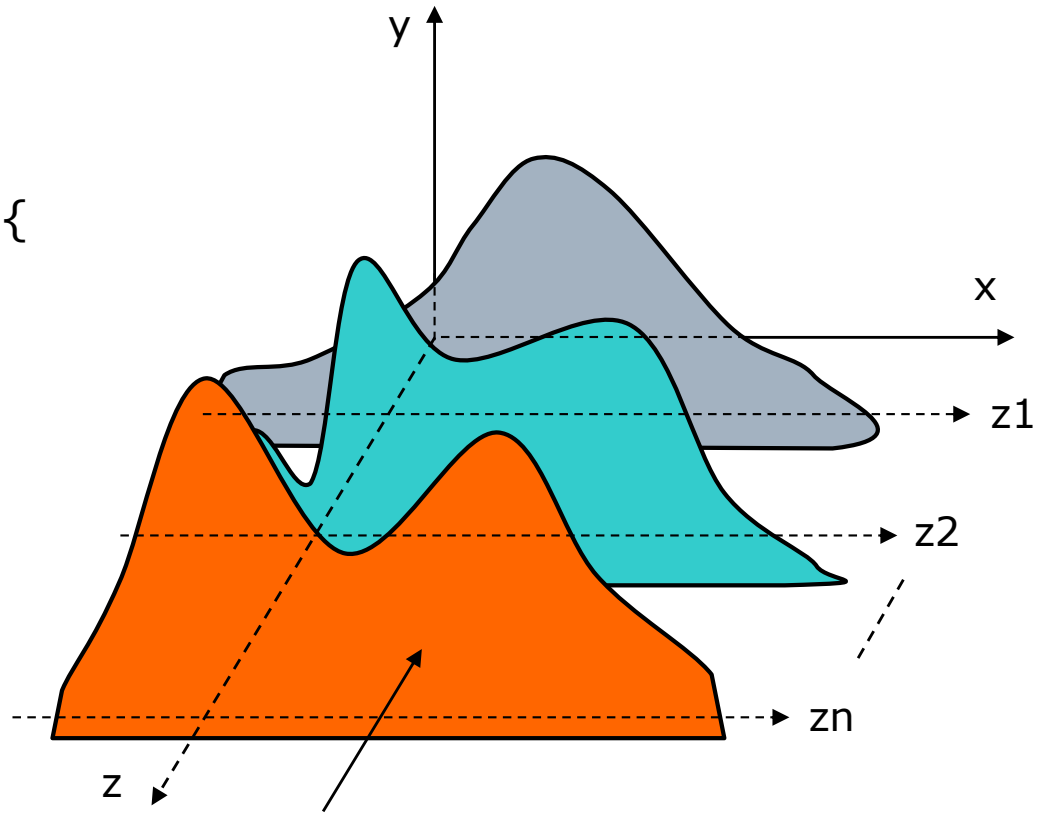
1. Floating Horizon Algorithm

- F(x,y,z)=0

  E.g., $x^2+y^2+z^2-r^2=0$

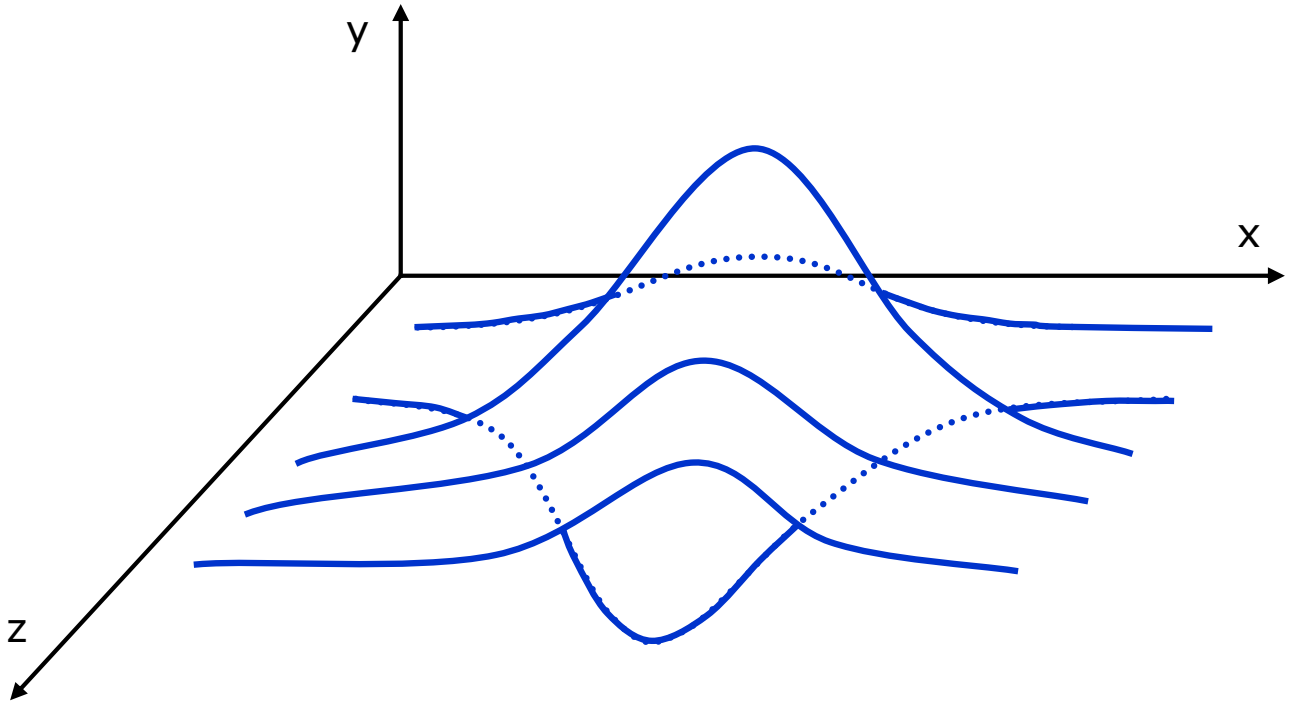
  z=zi, i=1,2,..,n

```
for z=z1 to zn {
        for x=xmin to xmax {
              render y=f(x,z)
        }
}
```

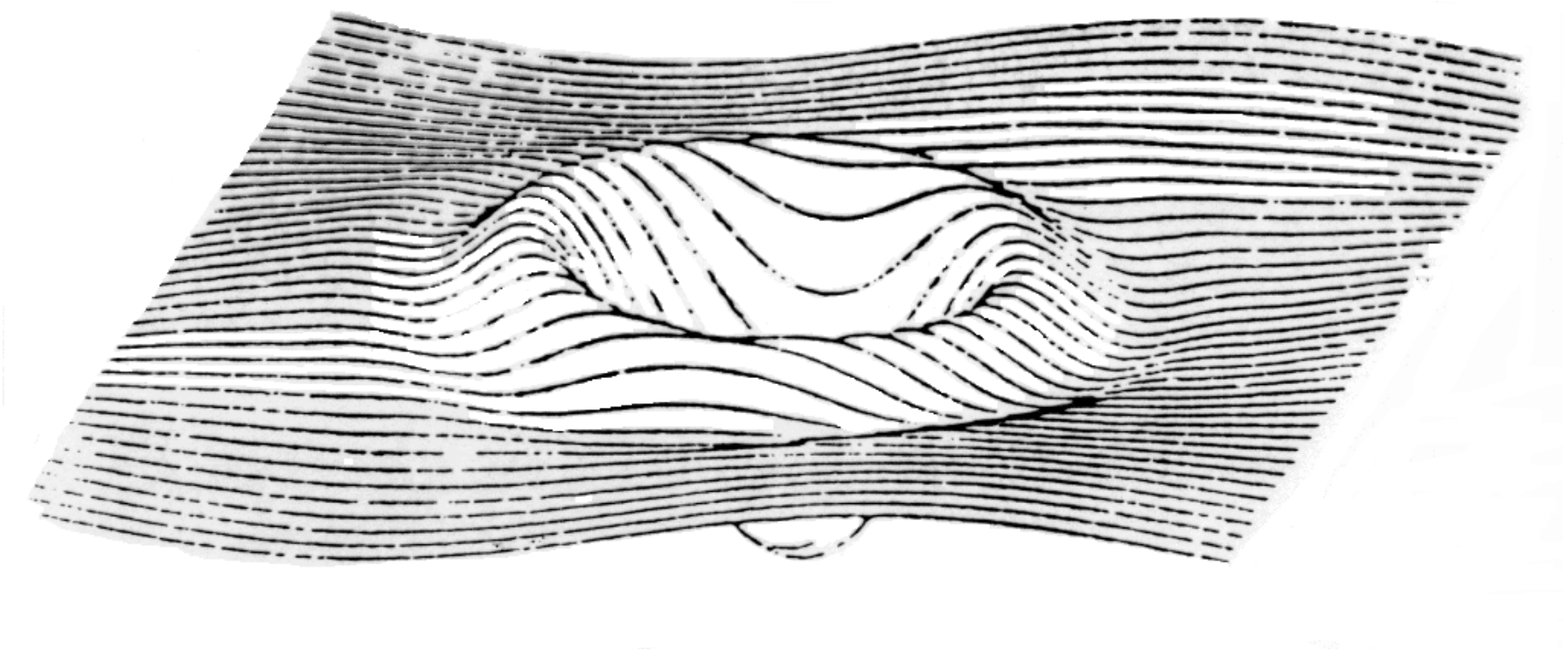


Functions in cutting planes

- Curves in cutting planes constant on z

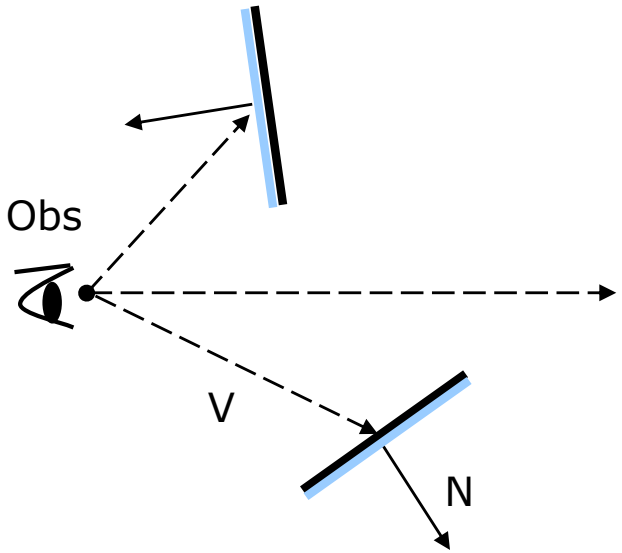


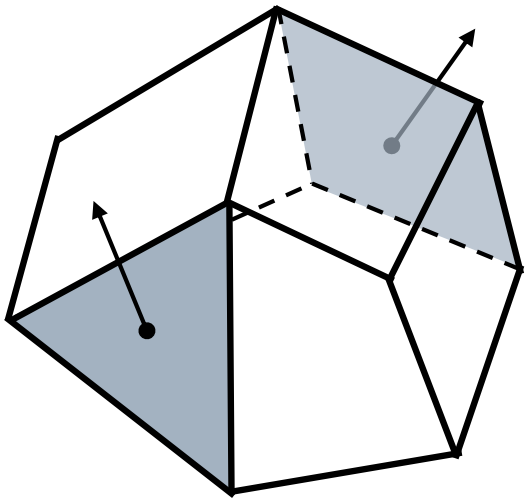
Floating Horizon Algorithm - example



2. Back-face culling algorithm

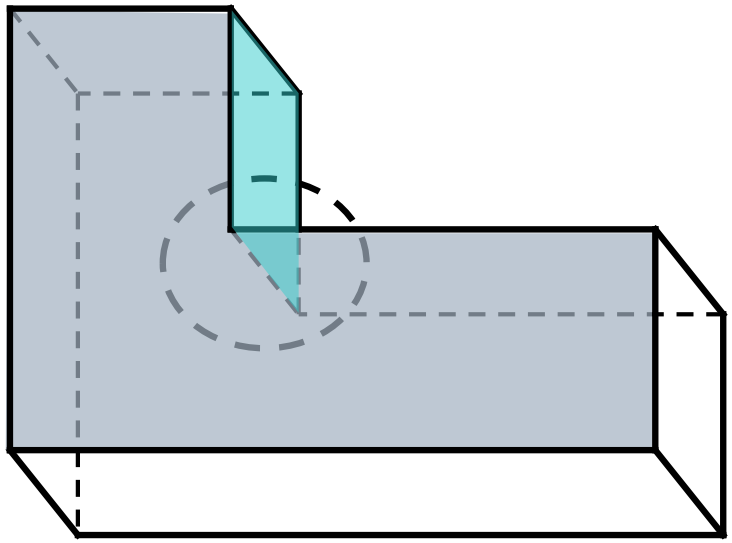
- A simple way to perform hidden surface removal by cutting off all the "backfacing" polygons
- "backfacing" polygon is the polygon for which the normal is facing away from the viewer
- For convex solid objects, the backfacing polygon are not visible by the viewer
- To compute the visibility of a polygon we use the dot product (see "point to plane and point to 3D convex object relationships" in a previous course "Mathematics in Computer Graphics") between normal vector N and vector V from viewer to polygon: $N \cdot V$
- if $N \cdot V > 0$, cull the polygon.





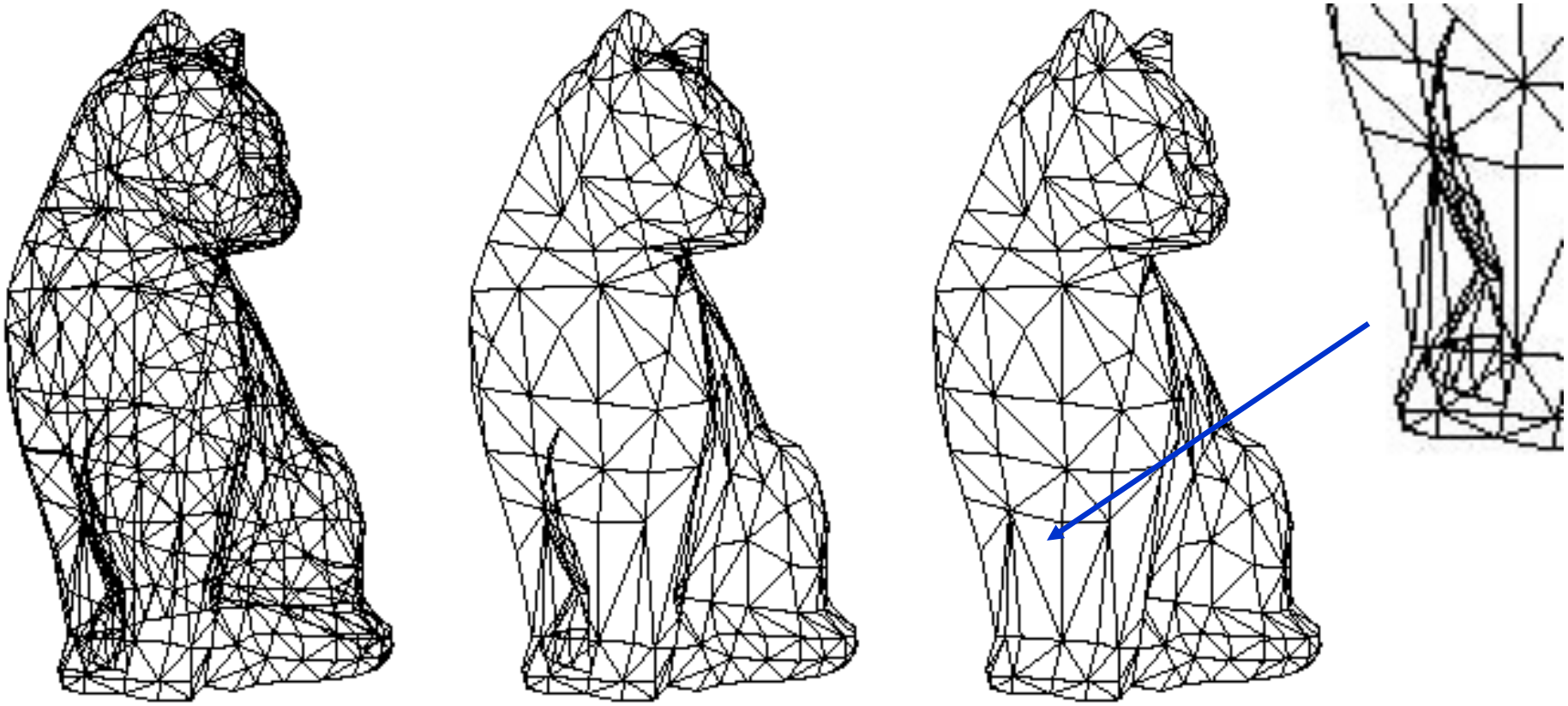
Back-face culling algorithm

- The back-face algorithm works well just for convex objects
- Usually works in conjunction with a more complete hidden surface algorithm
- Easy to integrate into hardware
- Usually improves performance by a factor of 2



Back-face and hidden algorithms

- Rendering by wireframe, back face culling, and hidden surface removal

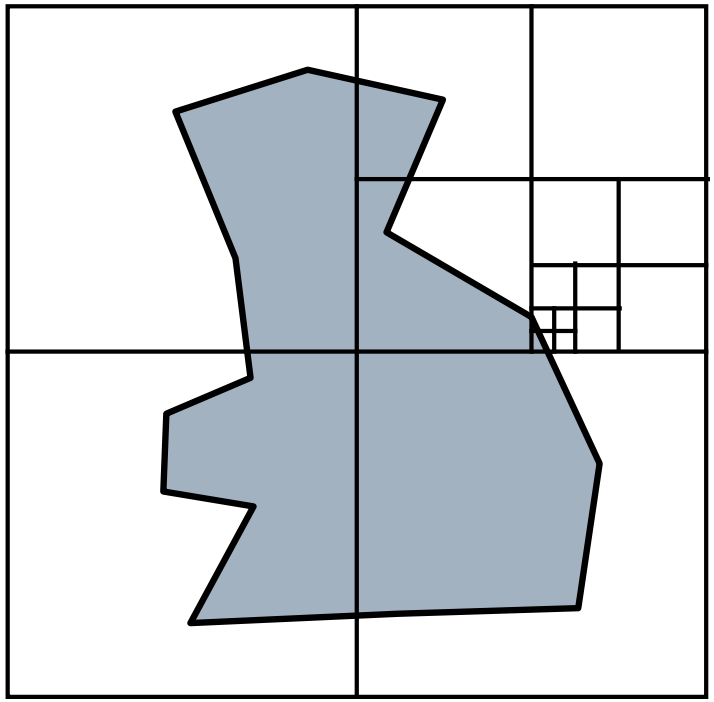


3. Area Subdivision Algorithms

- Warnock, 1969
- Catmull, 1974
- Weiler-Atherton, 1977

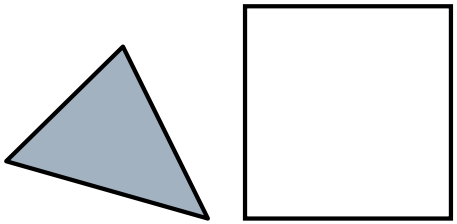
3.1 Warnock Algorithm (1969)

- Divide a window if any feature of interest (until pixel size)
- Consider the area coherence
- Work within image space
- For more polygons requires depth information
- Published in 1969, by Warnock, J., "A Hidden-Surface Algorithm for Computer Generated Half-Tone Pictures", Technical Report, Computer Science Department, Univ. of Utah.

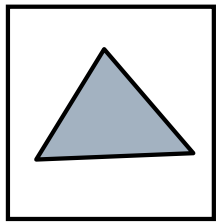


Polygon and elementary area

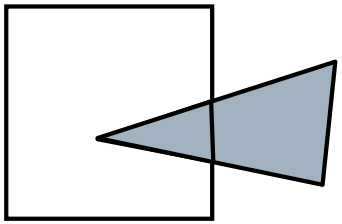
- There are 4 relationship types:



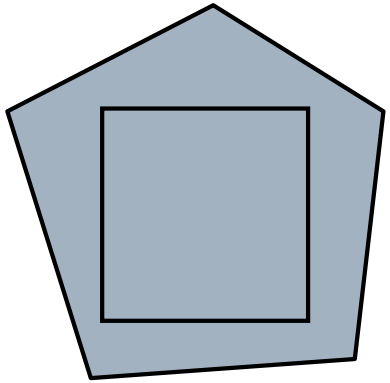
disjoint



contained



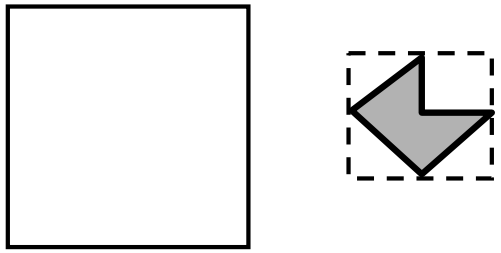
intersecting



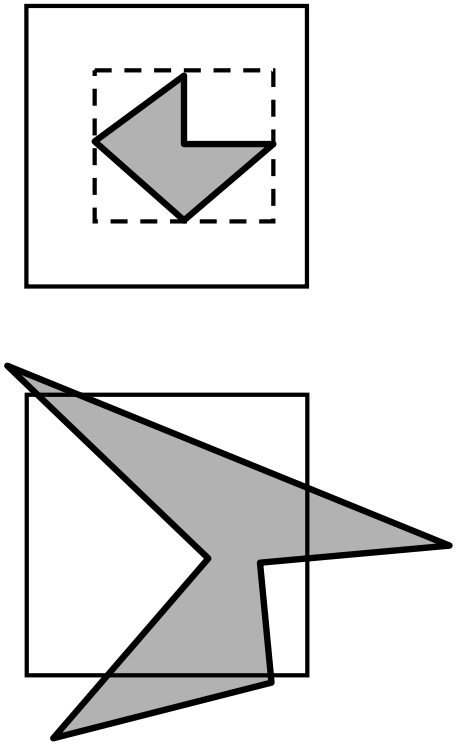
surrounding

Testing hierarchy

1. Box test
   - All the polygons are disjoint from the area. The background color can be displayed in the area

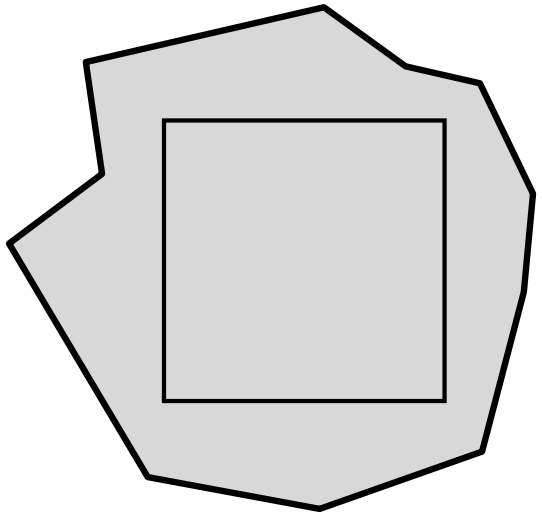


2. Intersection test
   - There is only one intersecting or only one contained polygon. The area is first filled with the background color, and then the part of the polygon contained in the area is scan-converted



Testing hierarchy

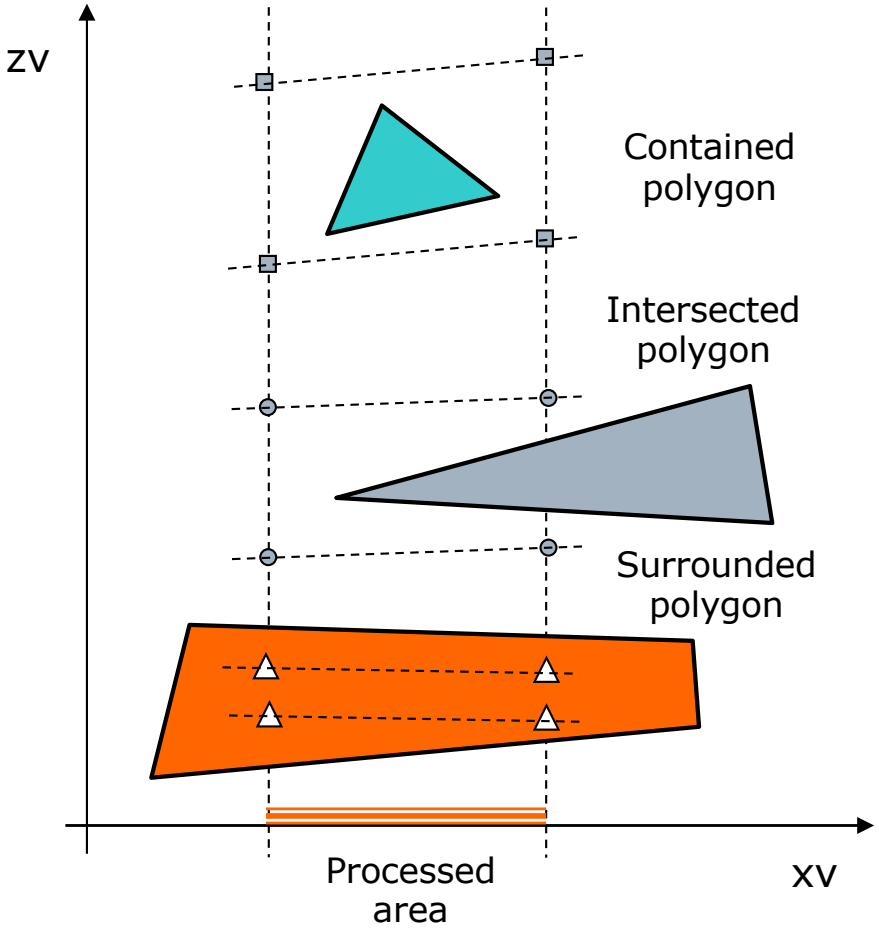
3. Surrounding test
   - There is a single surrounding polygon, but no intersecting or contained polygons. The area is filled with the color of the surrounding polygon

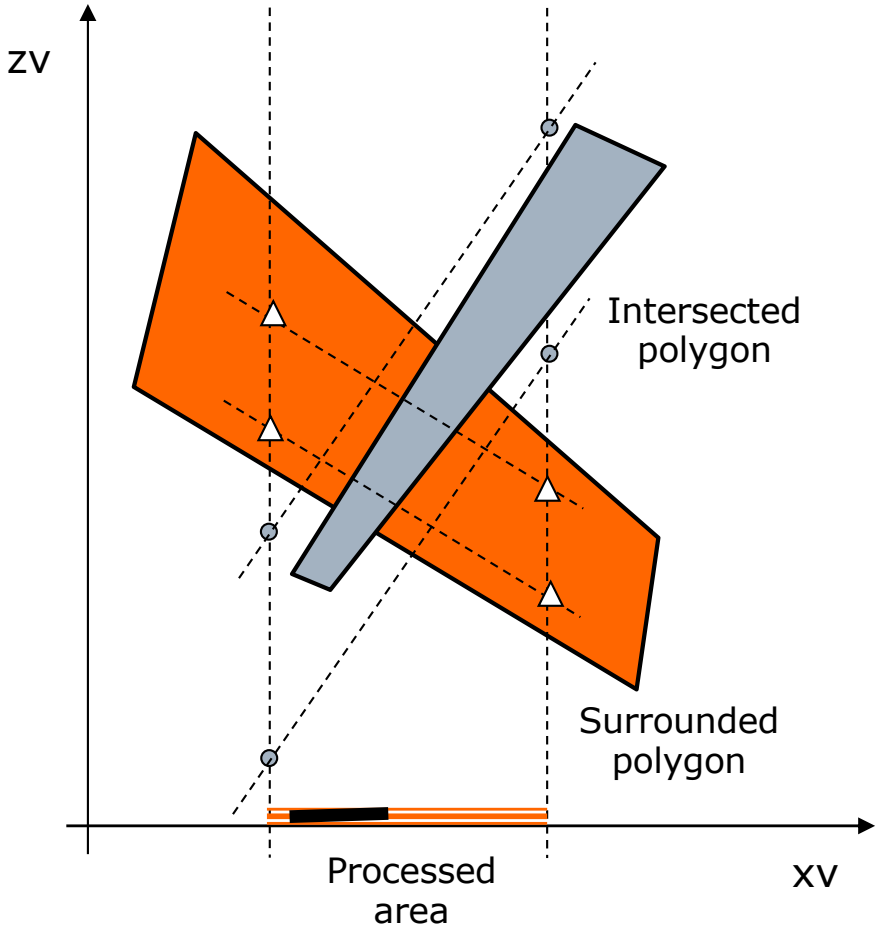


angle test

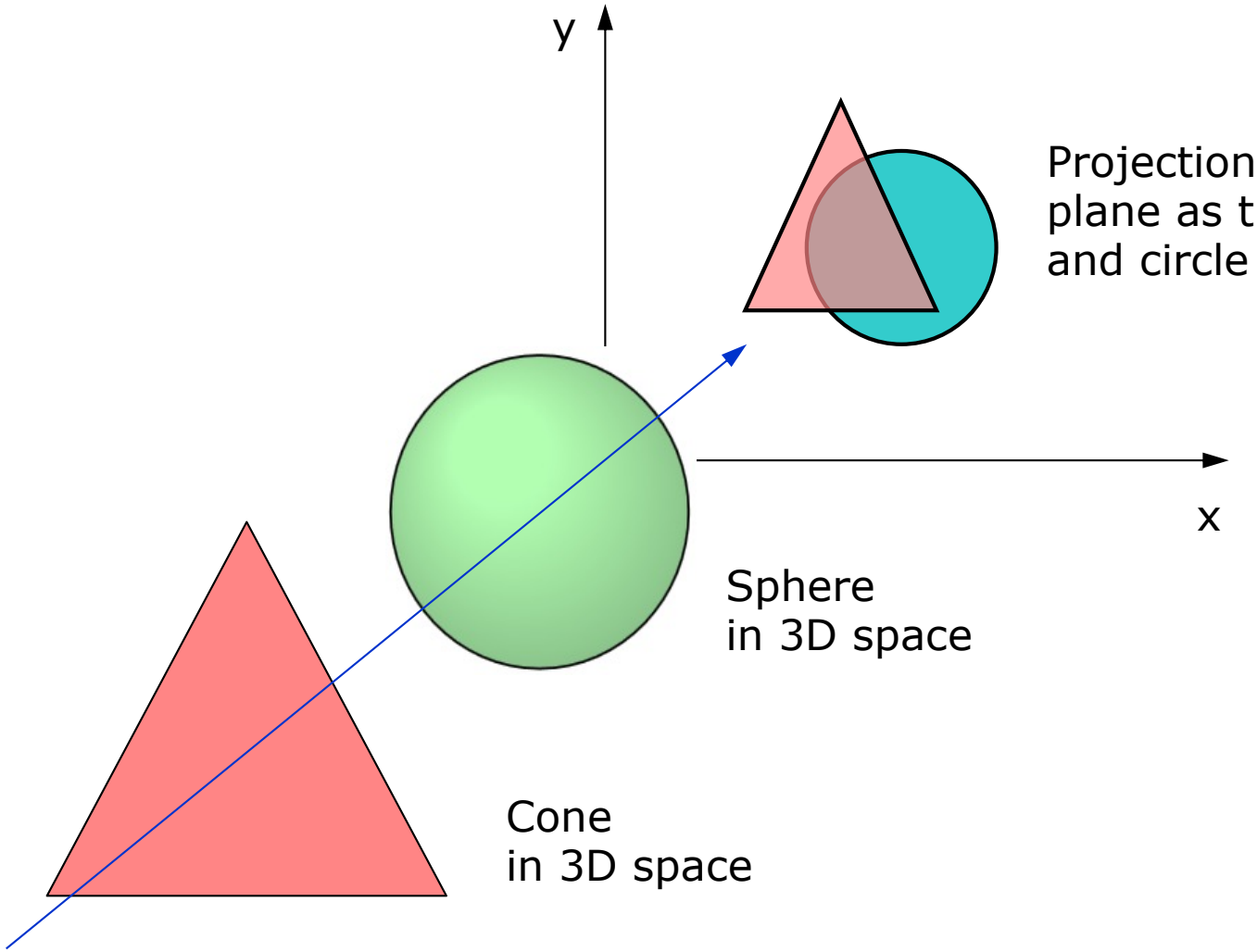
Testing hierarchy

4. More intersected polygons. One of them is a surrounding polygon and it is in front of all of the other polygons.

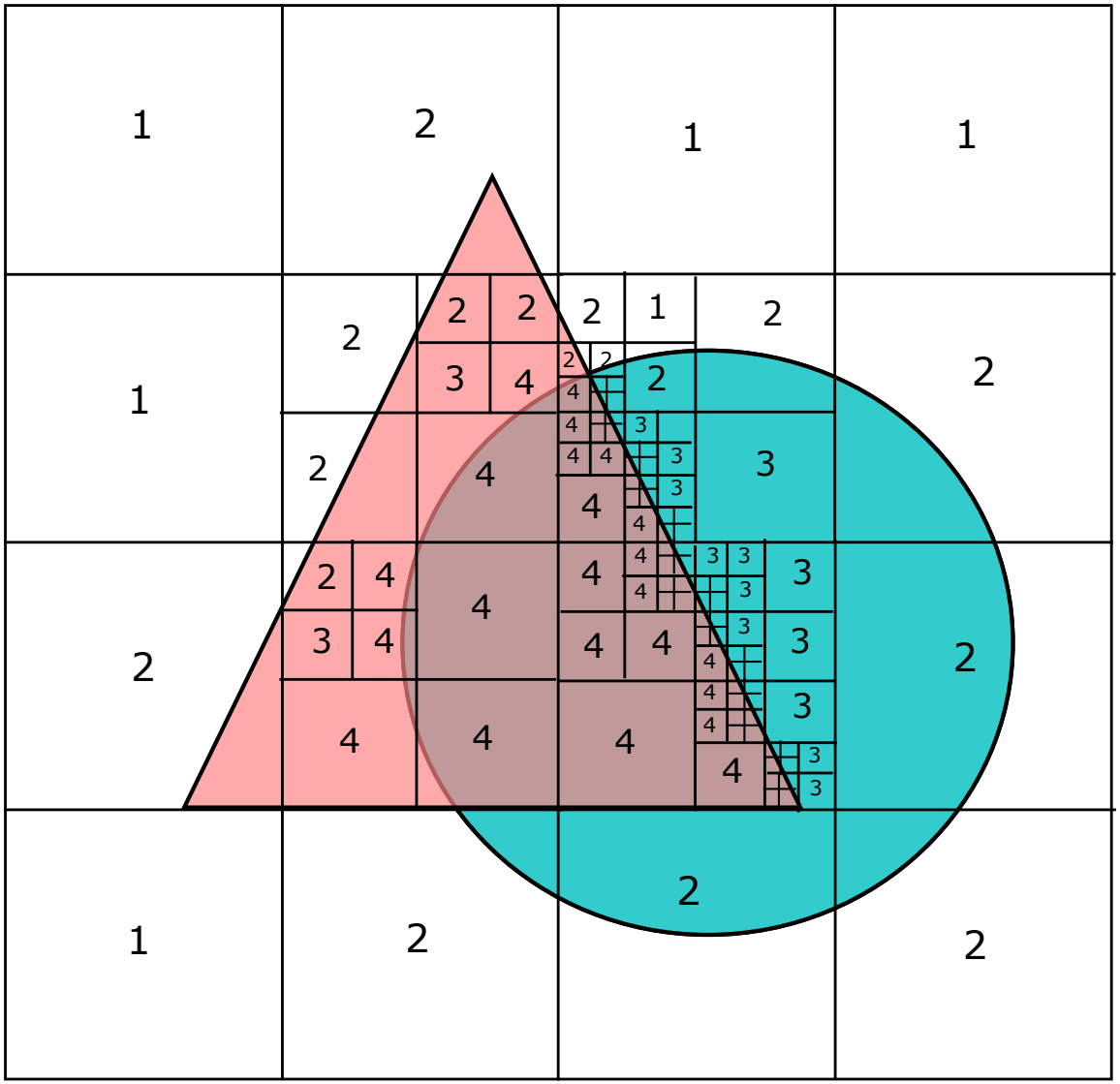




Warnock algorithm - example



Warnock algorithm - example



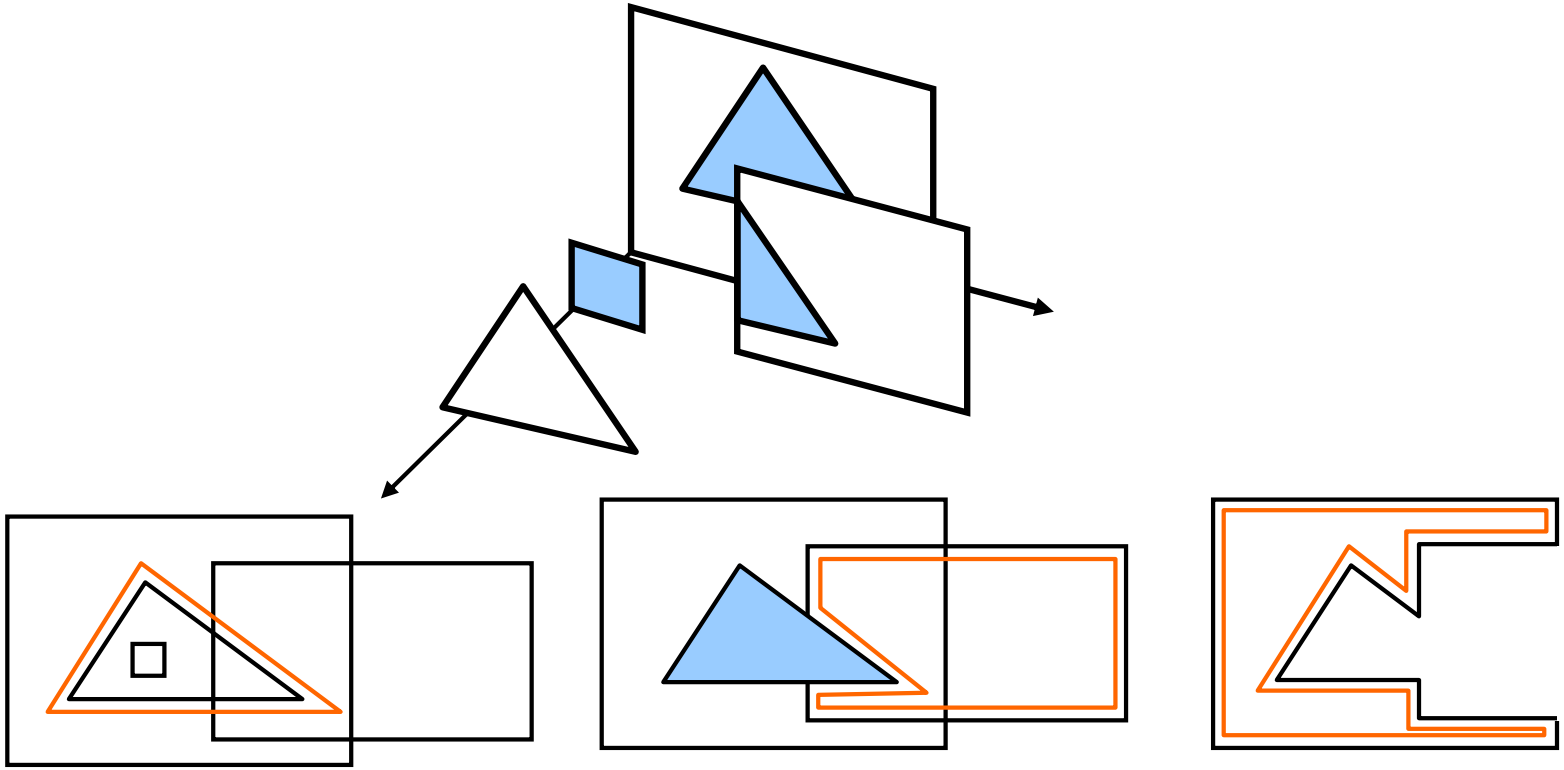
Warnock algorithm - description

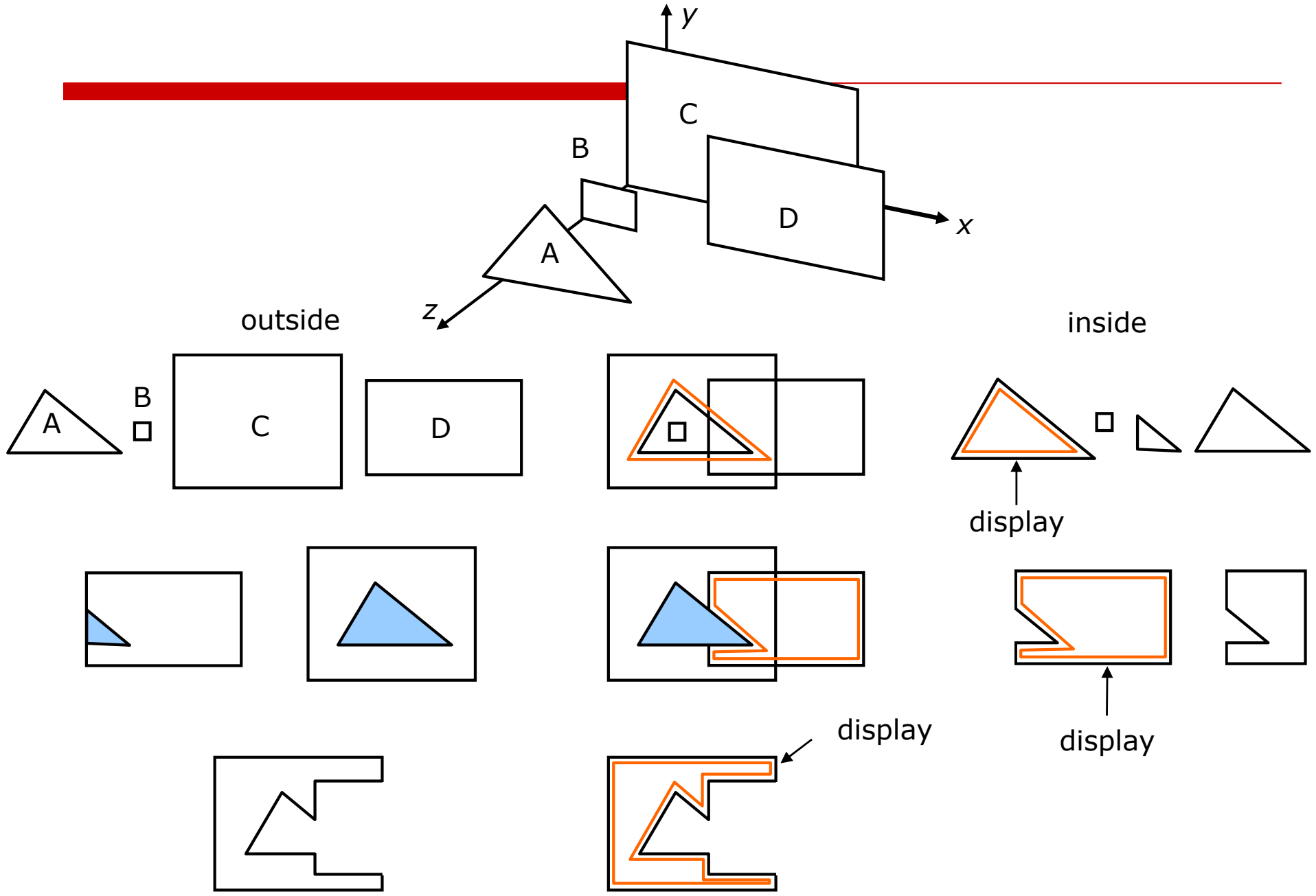
The input are details of polygons and a viewport.

1. Initialize the area
2. Create a list of polygons by sorting them by their Z values
3. Remove polygons that are outside the range
4. Identify the relationship of each polygon
5. Perform the visibility decision analysis:
    a. Fill the area with the background color if all polygons are disjunct
    b. Fill the entire area with background color and fill part of the polygon if there is only one polygon contained
    c. if there is a single surrounding polygon, but it is not included, then fill the area with the color of the surrounding polygon
    d. Sets the pixel to the color of the polygon that is closer to the area of the pixel (x, y) you want to view, and if none of (a) to (d) applies, calculates the z coordinate at the pixel (x, y) of the polygon
6. If none of the above points is correct, divide the area and continue with the second step

3.2 Weiler-Atherton Algorithm

- Minimize number of subdivisions in a Warnock-style algorithm.
- Published in 1977, by Weiler, K., and Atherton, P. “Hidden Surface Removal Using Polygon Area Sorting”, SIGGRAPH 77.



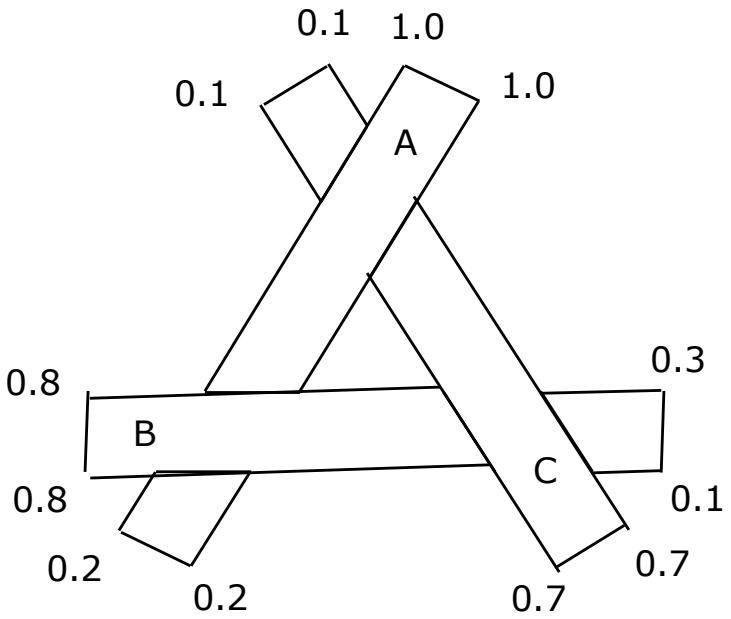

y
C
B
D
x
A
z
outside
inside
A
B
C
D
display
display
display

Weiler-Atherton Algorithm

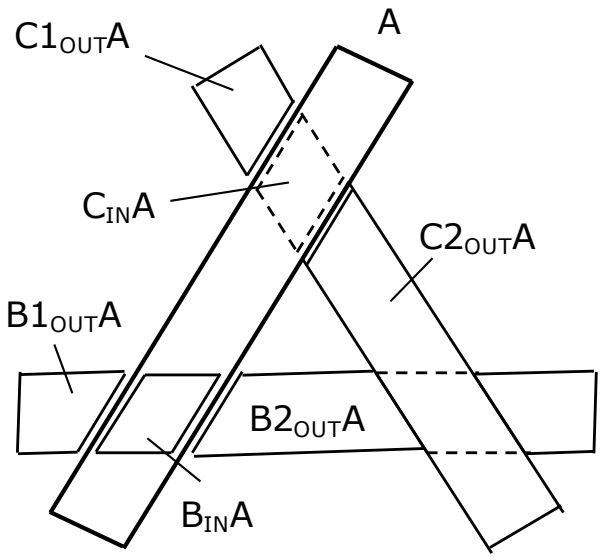
- Subdivides the screen area along polygon boundaries rather than along rectangle boundaries
- Requires the Weiler polygon clipping algorithm

- Basic approach:
  1. Sort polygons on some value of z, such as on their nearest z coordinate
  2. The closest polygon is used to clip all the polygons, including the clip polygon
  3. Result two lists of polygons: the list of the polygons inside and the list of the polygons outside the clip polygon
  4. All polygons on the inside list that are behind the clip polygon are then deleted, since they are invisible
  5. If any polygon on the inside list is closer than it is the clip polygon. It means the initial sort did not give a correct priority order
  6. Each such polygon is processed recursively to clip the pieces on the inside list against it
  7. When the recursive subdivision is over, the inside list is displayed
  8. The algorithm then continues to process the polygons on the outside list.

Weiler-Atherton Algorithm - example



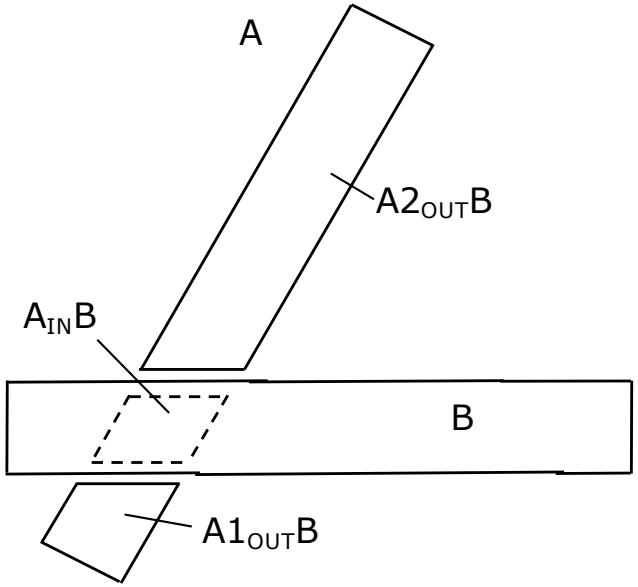
(a) Initially the scene contains the polygons A, B and C.



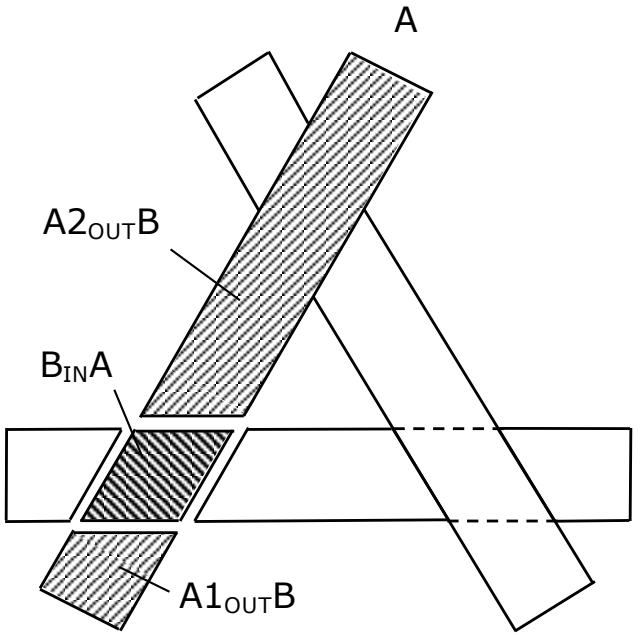
(b) Clipping against the polygon A.

- A is the clip polygon.
- A's inside list: A, $C_{IN}A$ and $B_{IN}A$
  - $C_{IN}A$ behind the A, then deleted. $B_{IN}A$ in front of A, then B the new clip polygon.
- A's outside list: $C1_{OUT}A$, $C2_{OUT}A$, $B1_{OUT}A$ and $B2_{OUT}A$.

Weiler-Atherton Algorithm - example



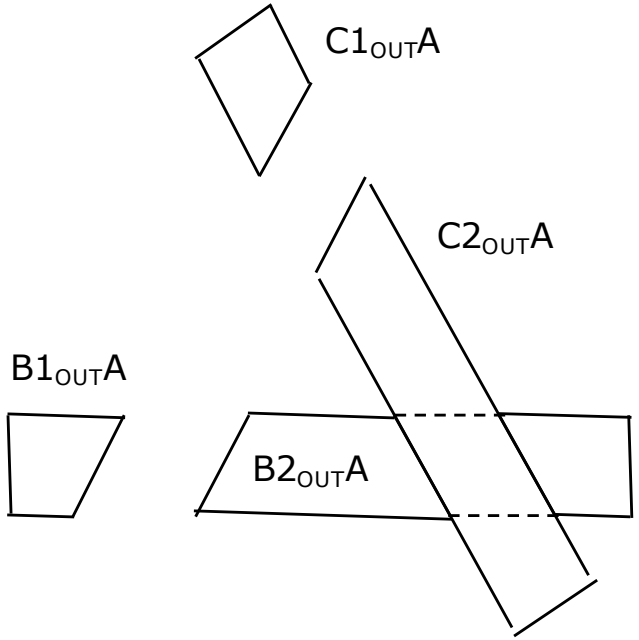
(c) Clipping the A's inside list against B.



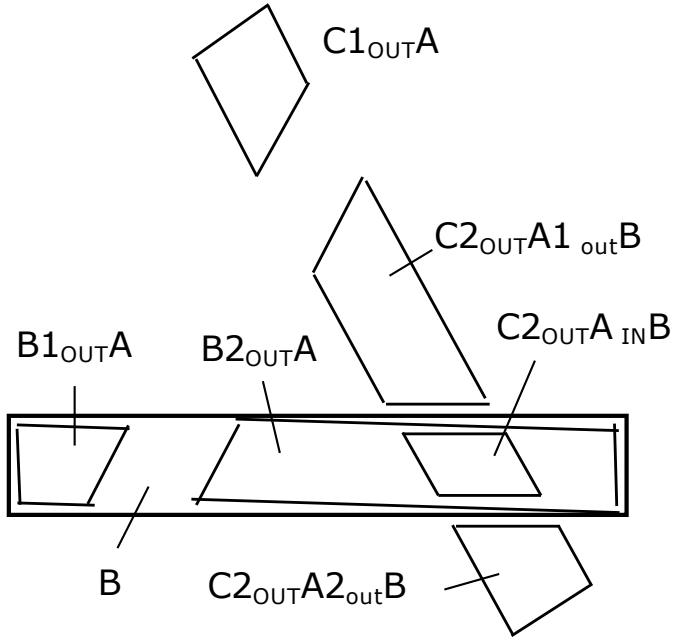
(d) Visible parts of the A's inside list.

- B is the new clip polygon for the A's inside list: A and $B_{IN}A$
- B's inside list: $A_{IN}B$ and $B_{IN}A$
  - $A_{IN}B$ behind B, then deleted. $B_{IN}A$ displayed.
- B's outside list: $A1_{OUT}B$ and $A2_{OUT}B$
  - Visible parts of the B's outside list: $A1_{OUT}B$ şi $A2_{OUT}B$

Weiler-Atherton Algorithm - example



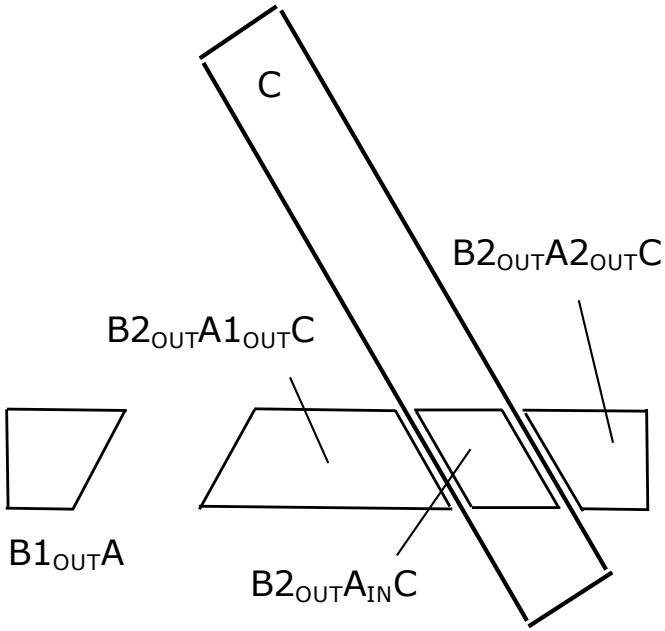
(e) A's outside list.



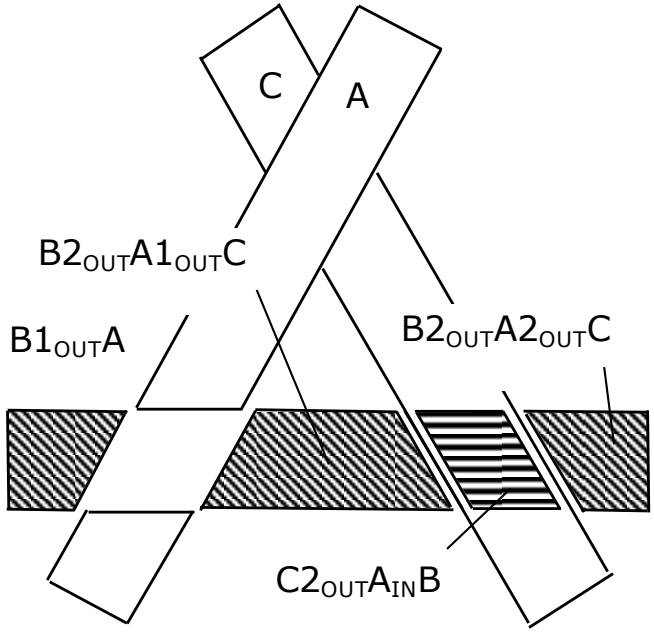
(f) Clipping the A's outside list against polygon B.

- B is the clip polygon for the A's outside list: $C1_{OUT}A$, $C2_{OUT}A$, $B1_{OUT}A$ şi $B2_{OUT}A$
- B's inside list: $B1_{OUT}A$, $C2_{OUT}A_{IN}B$ and $B2_{OUT}A$
  - $C2_{OUT}A_{IN}B$ in front of B, then the new clip polygon is C.
- B's outside list: $C1_{OUT}A$, $C2_{OUT}A1_{OUT}B$ şi $C2_{OUT}A2_{OUT}B$

Weiler-Atherton Algorithm - example



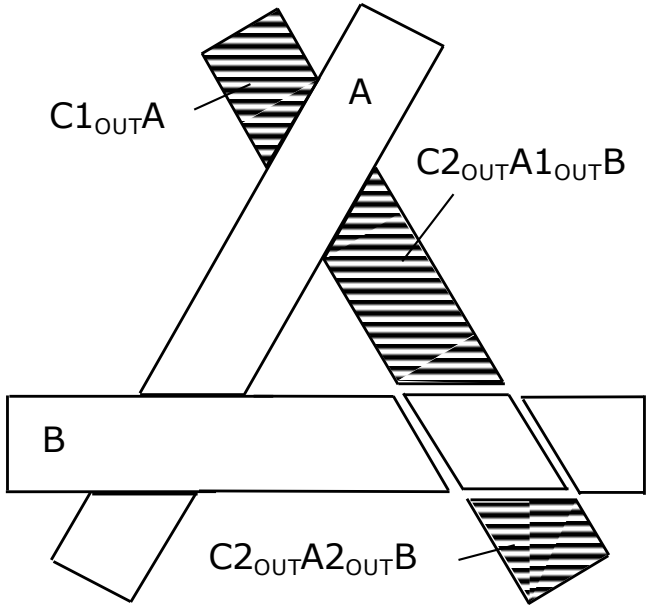
(g) Clipping the B's inside list against C.



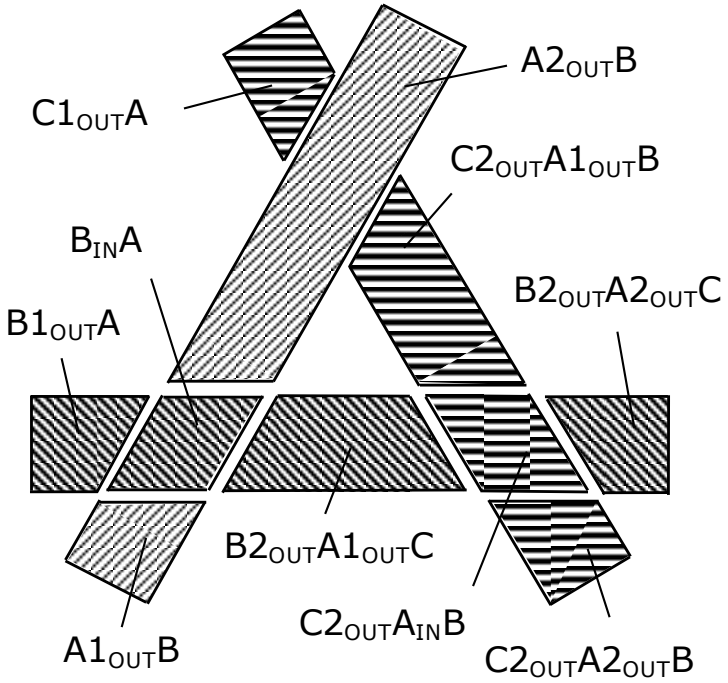
(h) Visible parts of the B's inside list.

- C is the clip polygon for the B's inside list: $B1_{OUT}A$, $C2_{OUT}A_{IN}B$ and $B2_{OUT}A$
- C's inside list: $C2_{OUT}A_{IN}B$ şi $B2_{OUT}A_{IN}C$
  - $B2_{OUT}A_{IN}C$ behind C, then it is deleted from the C's inside list. Displays $C2_{OUT}A_{IN}B$.
- C's outside list: $B1_{OUT}A$, $B2_{OUT}A1_{OUT}C$, $B2_{OUT}A2_{OUT}C$
  - Displays the whole C's outside list.

Weiler-Atherton Algorithm - example



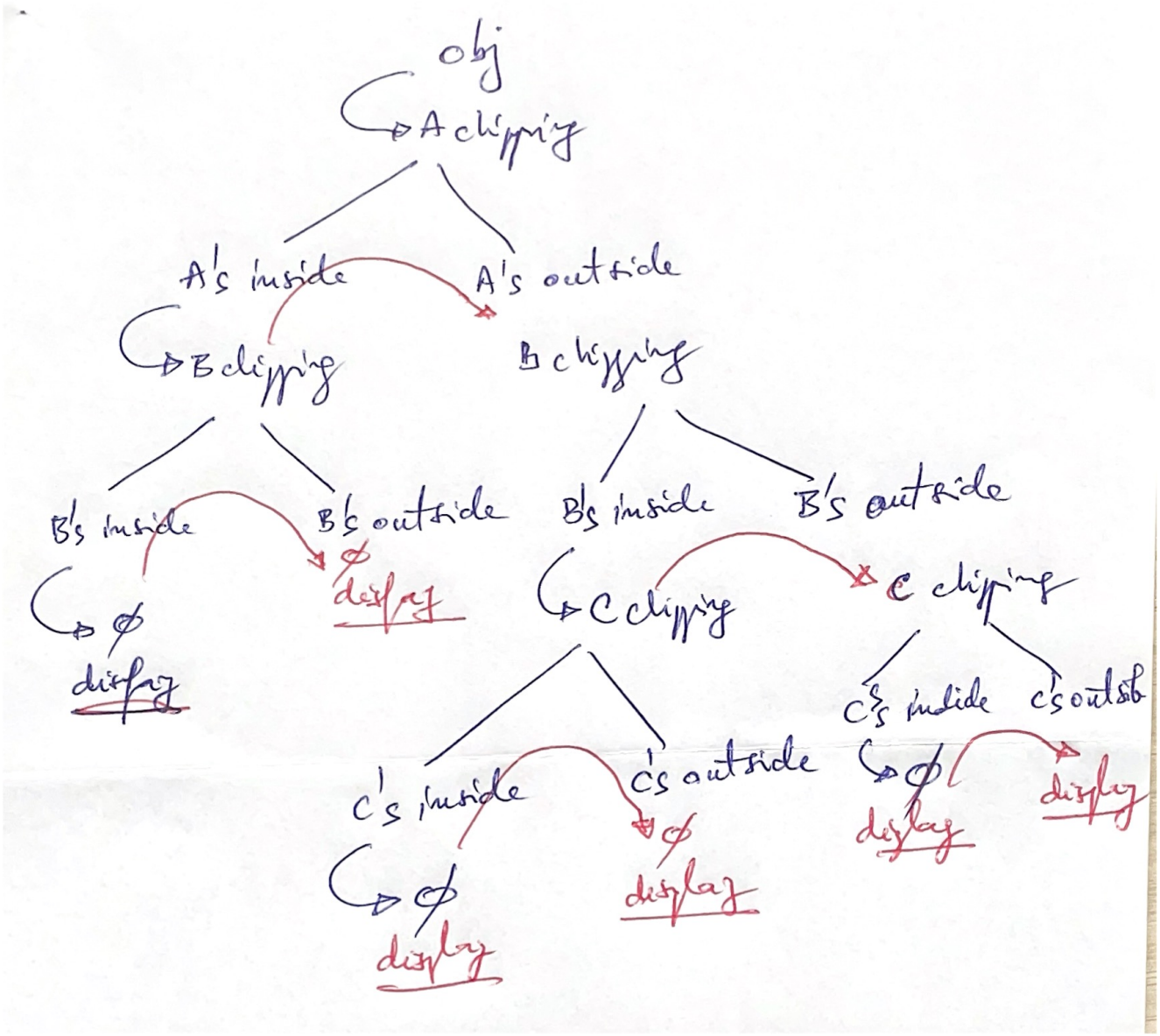
(i) Visible parts of the B's outside list.



(j) All visible parts displayed at completion.

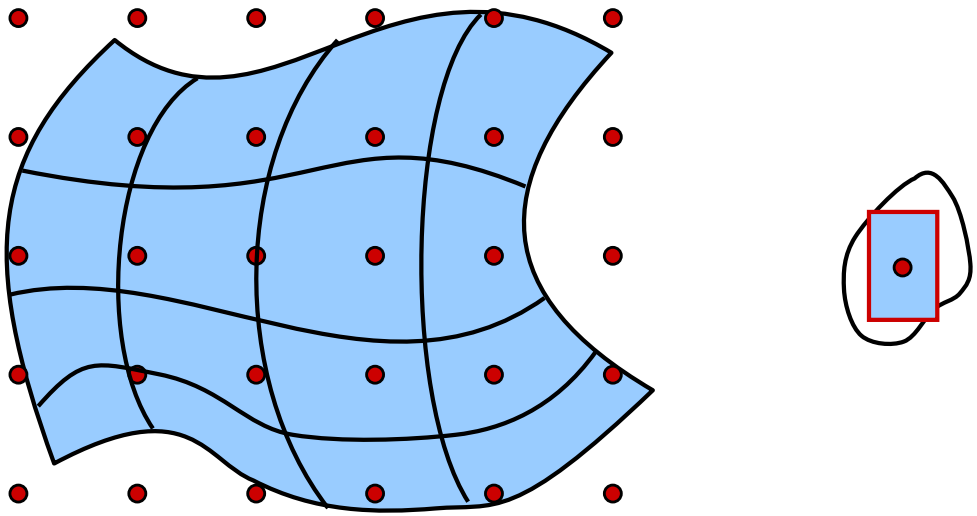
- Return from recursion and display the B's outside list:

  $C1_{OUT}A$, $C2_{OUT}A1_{OUT}B$ şi $C2_{OUT}A2_{OUT}B$.
- Finally display all visible pieces (j).


obj
A clipping
A's inside
A's outside
B clipping
B clipping
B's inside
B's outside
B's inside
B's outside
display
C clipping
C clipping
C's inside
C's outside
C's inside
C's outside
display
display
display
display

3.3 Catmull Algorithm (1974)

- Warnock-style
- Curved surfaces
- Subdivide surface (computations in object space rather than image space)
- Recursively subdivide surface into subpatches until a subpatch covers at most one pixel. A z-buffer algorithm determines whether the patch is visible at this pixel
- Compute the attribute of the surface at the pixel and display if needed.



Catmull algorithm - pseudocode

- Published in 1974, by Catmull, E., "A Subdivision Algorithm for Computing Display of Curved Surfaces", PhD Thesis, Univ. of Utah.
- Catmull recursive-subdivision algorithm:

```
for each patch {
    push patch onto stack;
    while stack not empty {
        pop patch from stack;
        if patch covers ≤ 1 pixel {
            if patch's pixel closer in z then
                determine shade and draw
        } else {
            subdivide patch into 4 subpatches;
            push subpatches onto stack
        }
    }
}
```

4. Depth-Buffer Algorithms

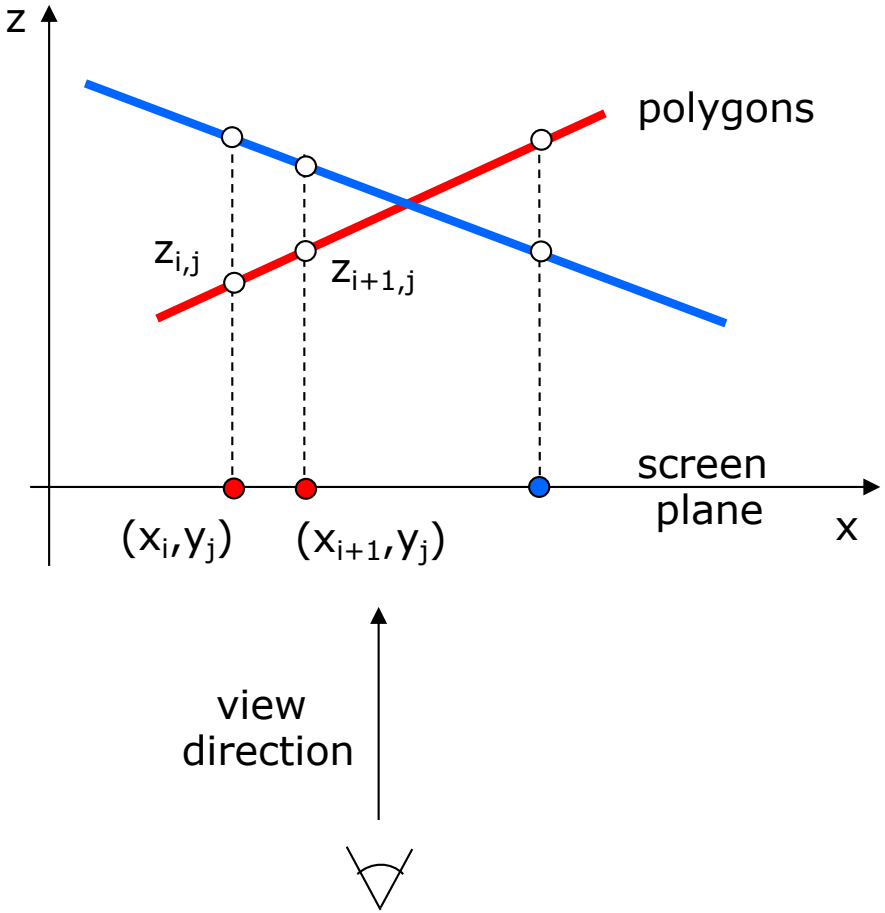
- Called **Z-buffer** or depth buffer, developed by Catmull, in 1974.
- Published in 1974, by Catmull, E., "A Subdivision Algorithm for Computing Display of Curved Surfaces", PhD Thesis, Univ. of Utah.
- Works with two buffers:
  1. frame buffer F – stores the color values, builds up the image
  2. Z-buffer Z – with the same number of entries, in which a z-value is stored for each pixel
- Z-buffer is initialized to the largest value that can be stored and represents the z of the far clipping plane (see the frustum definition)
- Frame buffer is initialized to the background color
- Polygons are scan-converted into the frame buffer in arbitrary order
- During the scan-conversion process, if the polygon point being scan-converted at (x,y) is closer from the viewer than is the point whose color and depth are currently in the buffers, than the new point's color and depth replace the old values
- Complexity
  - Time : nxm buffer, k poygons -> O(nmk)
  - Space : b depth precision -> O(nmb)

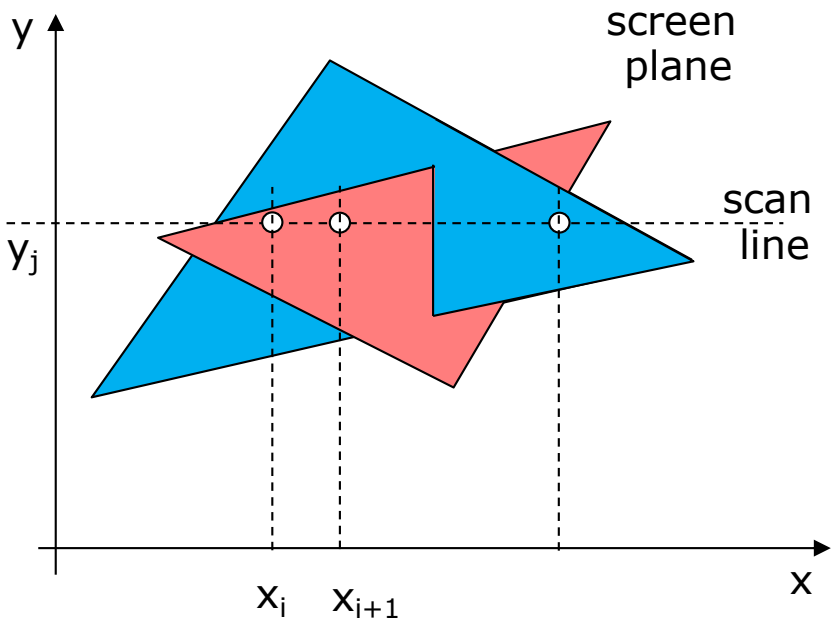
z-buffer algorithm - pseudocode

```
function zBuffer {
    int pz;
    for y=0 to YMAX {
        for x=0 to XMAX {
            WritePixel(x, y, background_value);
            WriteZ(x, y, ZMAX)
        }
    }

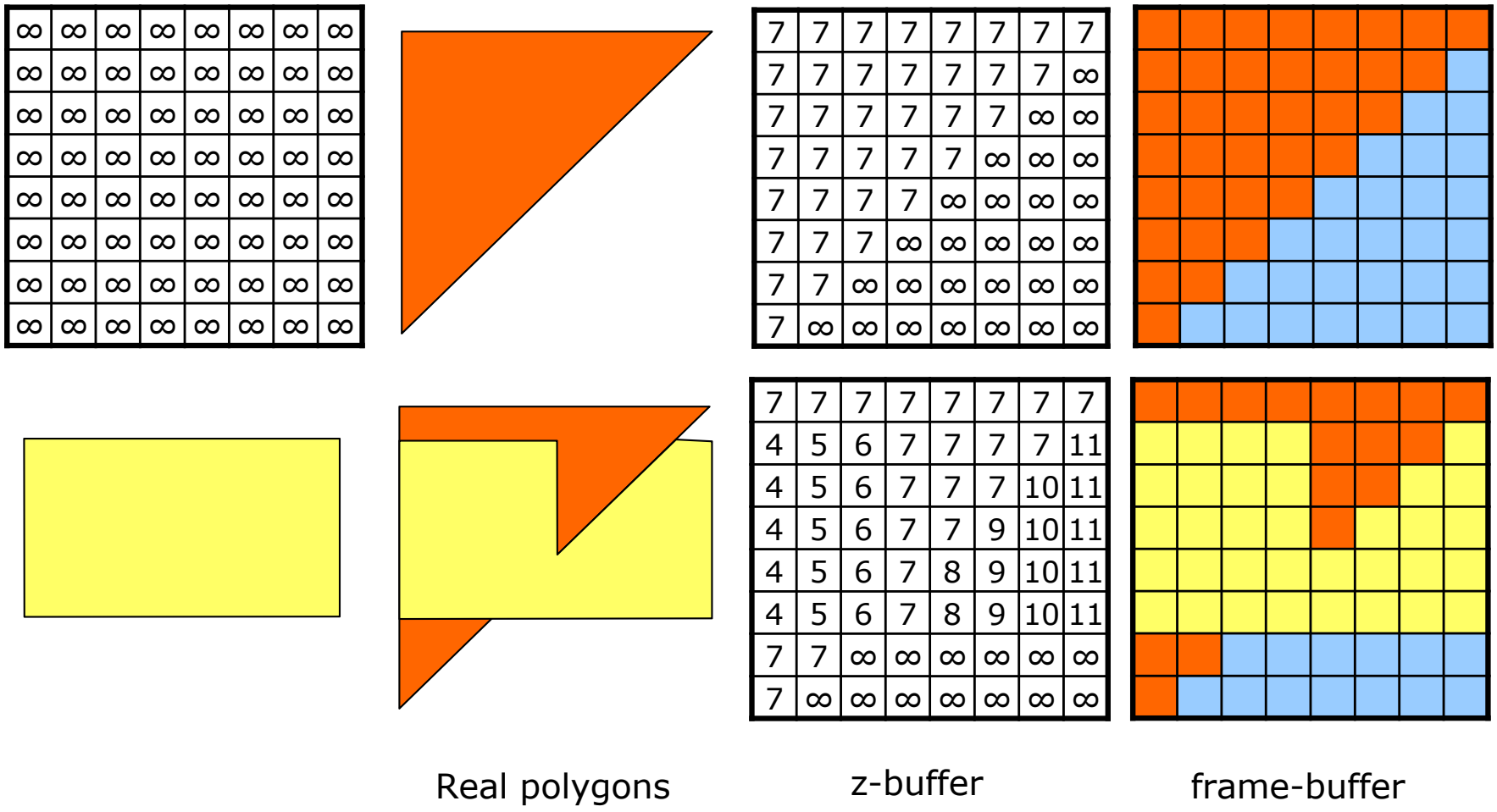
    for each polygon {
        for each pixel in polygon's projection {
            pz=polygon's z-value at pixel coords (x,y);
            if pz ≤ ReadZ(x, y) {
                WriteZ(x, y, pz)
                WritePixel(x, y, polygon's color at pixel coords (x, y))
            }
        }
    }
}
```

z-buffer algorithm





z-buffer algorithm - example



Real polygons z-buffer frame-buffer

z-buffer algorithm – depth coherence

Considering three vertices of the polygonal face:

V1(x1,y1,z1), V2(x2,y2,z2), and V3(x3,y3,z3)

The equation of the polygonal face can be computed (see the course "Mathematics in Computer Graphics), and looks like:

$$Ax+By+Cz+D=0$$

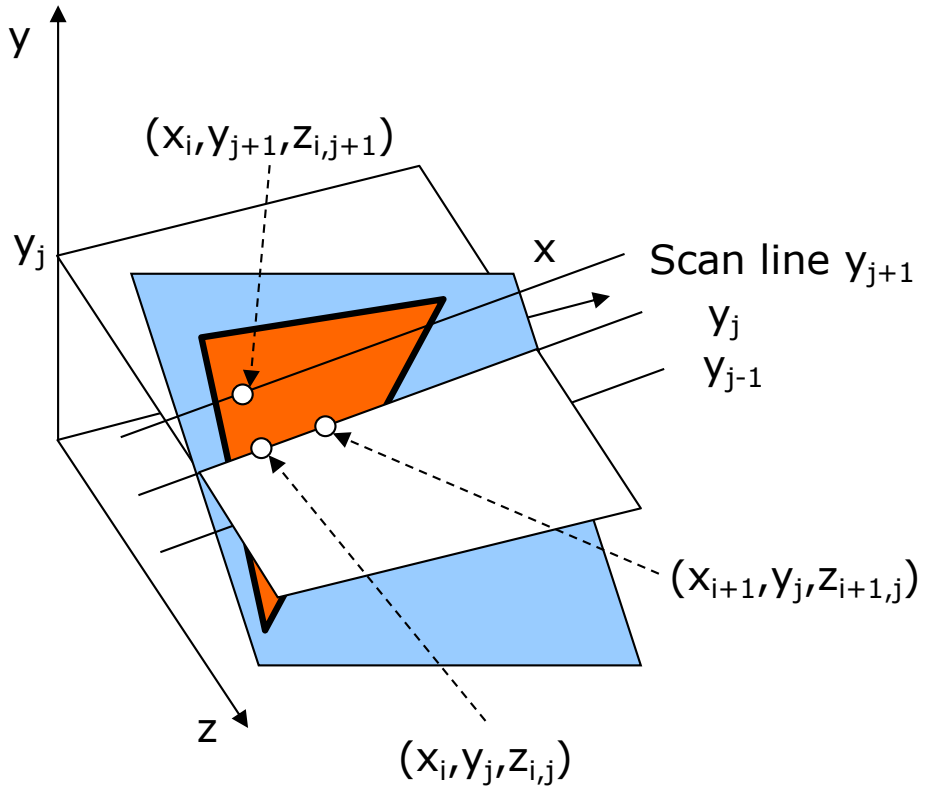
$$z_{i,j} = \frac{-D - Ax_i - By_j}{C}$$

$$z_{i+1,j} = \frac{-D - Ax_{i+1} - By_j}{C}$$

$$x_{i+1} = x_i + 1,\ y_{j+1} = y_j + \Delta y$$

$$z_{i+1,j} = z_{i,j} - \frac{A}{C}$$

$$z_{i,j+1} = z_{i,j} - \frac{B\,\Delta y}{C}$$



Questions and problems

1. Enunciate and explain two reasons for using the hidden line and surface removal algorithms mainly in one space instead of another (e.g. in the view space rather than in the screen space).
2. Exemplify and explain the concept of depth coherence.
3. Why the hidden line and surface removal algorithms work faster in the case of orthogonal projection than the case of perspective projection?
4. Why the back face culling algorithm works better for convex 3D objects?
5. Why throughout the Warnock algorithm is using the classification of the relationship between the projected polygon and the elementary area?
6. Exemplify and explain the Warnock algorithm in the visualization of: (a) just one triangle; (b) two triangles; (c) one cube.

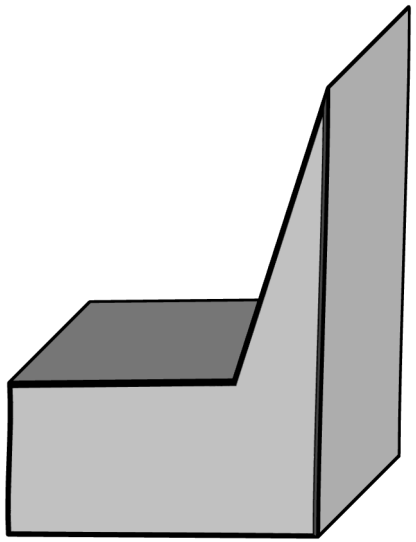
Questions

7. Specify, relative to the right hand coordinate system, one example of the object's position and orientation, and viewer's position, to get on the the XY plane the image like in the figure 1, by orthogonal projection.

8. Exemplify and explain the Weiler-Artherton algorithm for the visualization of a cube.

9. Exemplify and explain the Weiler-Artherton algorithm, for the hidden line and surface removal, for the 3D object in the figure 1. The resulted image by orthogonal projection is like in the figure.

Figure 1.



Questions

10. Exemplify and explain the z-Buffer algorithm for the visualization of a cube.

11. Exemplify and explain the z-Buffer algorithm, for the hidden line and surface removal, for the 3D object in the figure 1. The resulted image by orthogonal projection is like in the figure.

Figure 1.

